

Vidya Academy of Science & Technology

(A unit of Vidya International Charitable Trust)

Approved by AICTE & Affiliated to APJ Abdul Kalam Technological

University Accredited by NAAC with "B++" grade



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

PCCSL508 MACHINE LEARNING LAB

LAB MANUAL

PREPARED BY:

Dr. Arjun K P

APPROVED BY:

HOD, CSE

Vision of the Institution

To be a premier institution in engineering education and research, nurturing innovative professionals for sustainable societal development.

Mission of the Institution

- M1.** Provide quality engineering education that empowers students with strong technical competence and analytical problem solving abilities.
- M2.** Promote research, innovation, entrepreneurship and industry collaboration to develop sustainable technological solutions.
- M3.** Develop ethical and socially responsible professionals with leadership qualities, lifelong learning abilities and a commitment to societal well-being.

Vision of the Department

To excel in Computer Science and Engineering education and research, nurturing innovative and ethically responsible professionals for sustainable technological development.

Mission of the Department

- M1.** Impart quality education in Computer Science and Engineering by providing strong theoretical foundations, practical skills and exposure to emerging technologies.
- M2.** Promote research, innovation, entrepreneurship and industry collaboration to develop sustainable and socially relevant computing solutions.
- M3.** Develop graduates with professional competence, ethical values, logical and analytical thinking skills, leadership qualities and a commitment to lifelong learning and societal well-being.

Program Educational Objectives (PEOs)

PEO1: Graduates will be able to apply strong foundations in Computer Science and Engineering to design and develop software solutions using modern tools and technologies

PEO2: Graduates will engage in research, innovation and entrepreneurship develop sustainable and industry-relevant computing solutions.

PEO3: Graduates will demonstrate professional ethics, leadership and effective teamwork while contributing responsibly to society through their engineering practice.

PEO4: Graduates will pursue continuous professional development and lifelong learning to remain proficient in evolving technologies throughout their careers.

Program Specific Outcomes (PSOs)

PSO1: Build robust software systems and intelligent applications using modern programming languages, frameworks and algorithms.

PSO2: Apply core concepts in networking and cloud computing to design efficient and secure systems for real-world applications

PSO3: Develop innovative, sustainable and socially relevant software solutions through interdisciplinary collaboration and industry engagement.

Program Outcomes (POs)

Computer Science & Engineering graduates will be able to:

PO1: Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.

PO2: Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)

PO3: Design/Development of Solutions: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with

consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)

PO4:Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8).

PO5:Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)

PO6:The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7).

PO7:Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)

PO8:Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.

PO9:Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences

PO10:Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.

PO11:Life-Long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change. (WK8)

Sustainable Development Goals (SDGs)

SDG 1 : No Poverty: End poverty in all its forms everywhere.

SDG 2 : Zero Hunger: End hunger, achieve food security and improved nutrition and promote sustainable agriculture.

SDG 3 : Good Health and Well-being: Ensure healthy lives and promote well-being for all at all ages.

SDG 4 : Quality Education: Ensure inclusive and equitable quality education and promote lifelong learning opportunities for all.

SDG 5 : Gender Equality: Achieve gender equality and empower all women and girls.

SDG 6 : Clean Water and Sanitation: Ensure availability and sustainable management of water and sanitation for all.

SDG 7 : Affordable and Clean Energy: Ensure access to affordable, reliable, sustainable and modern energy for all.

SDG 8 : Decent Work and Economic Growth: Promote sustained, inclusive and sustainable economic growth, full and productive employment and decent work for all.

SDG 9 : Industry, Innovation and Infrastructure: Build resilient infrastructure, promote inclusive and sustainable industrialization and foster innovation.

SDG 10 : Reduced Inequalities: Reduce inequality within and among countries.

SDG 11 : Sustainable Cities and Communities: Make cities and human settlements inclusive, safe, resilient and sustainable.

SDG 12 : Responsible Consumption and Production: Ensure sustainable consumption and production patterns.

SDG 13 : Climate Action : Take urgent action to combat climate change and its impacts.

SDG 14 : Life Below Water : Conserve and sustainably use the oceans, seas and marine resources for sustainable development.

SDG 15 : Life on Land : Protect, restore and promote sustainable use of terrestrial ecosystems, sustainably manage forests, combat desertification, and halt and reverse land degradation and halt biodiversity loss.

SDG 16 : Peace, Justice and Strong Institutions: Promote peaceful and inclusive societies for sustainable development, provide access to justice for all and build effective, accountable and inclusive institutions at all levels.

SDG 17 : Partnerships for the Goals: Strengthen the means of implementation and revitalize the global partnership for sustainable development.

COURSE INFORMATION

Program: B. Tech	Department: Computer Science & engineering
Course: Machine Learning Lab	Semester: S5 Credits: 2
Course Code: PCCSL 508	Regulation: 2024
Course Area/Domain: Data Science/AIML	L: T: TW/SL: 3 P: 3
Corresponding Lab Course Code (If Any):	Lab Course Name:

COURSE PRE-REQUISITES:

CODE	COURSE NAME	DESCRIPTION	SEM
PCCSL 508	MACHINE LEARNING LAB	MACHINE LEARNING	S5

COURSE OUTCOMES:

SL NO	DESCRIPTION
CO1	Understand complexity of Machine Learning algorithms and their limitations.
CO2	Understand modern notions in data analysis-oriented computing.
CO3	Apply common Machine Learning algorithms in practice and implement their own.
CO4	Performing experiments in Machine Learning using real-world data.

CO - PO/PSO MAPPING:

CO Number	PSO STRENGTH													
	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO1 0	PO1 1	PSO 1	PSO 2	PSO 3
CO1	3	3	3		3						3	3	3	
CO2	3	3	3		3						3	3	3	
CO3	3	3	3		3						3	3	3	
CO4	3	3	3		3						3	3	3	
Total	3	3	3		3						3	3	3	

COURSE OBJECTIVES

To give the learner a practical experience of the various machine learning techniques and be able to demonstrate them using a language of choice (Python).

GENERAL INSTRUCTIONS (for the lab)

1. Install Python 3.x along with the required libraries: numpy, pandas, matplotlib, seaborn, scikit-learn, tensorflow/keras, scipy.
2. Carry your lab record and get it certified after every session.
3. Ensure datasets are downloaded/prepared prior to the lab session wherever direct sklearn access is not available.
4. Understand the theory behind each algorithm before implementing it — do not just run the code.
5. Record observations, output screenshots/plots, and inferences for every experiment.
6. Be prepared to answer viva-voce questions related to the algorithm, implementation, and evaluation metrics used.

COURSE SYLLABUS

List of Experiments

1. Linear Regression with One Variable — California Housing Dataset
2. Polynomial Regression — Auto MPG Dataset
3. Ridge and Lasso Regression — Diabetes Dataset
4. Logistic Regression — MLE and MAP Estimation on Breast Cancer Wisconsin Dataset
5. MLE and MAP for Multinomial Distribution — 20 Newsgroups Dataset
6. Logistic Regression with and without Feature Scaling — Pima Indians Diabetes Dataset
7. Naive Bayes Text Classification — 20 Newsgroups Dataset (Multinomial vs Bernoulli)
8. K-Nearest Neighbors (KNN) for Image Classification — Fashion MNIST Dataset
9. Decision Tree (ID3) for Customer Segmentation — Online Retail Dataset
10. Logistic Regression vs Decision Trees — Adult Income Dataset
11. Linear Support Vector Machine (SVM) — Iris Dataset
12. SVM Kernel Comparison (Linear, Polynomial, RBF) Fashion MNIST
13. Dataset Multilayer Feed-Forward Network (MLP) — Wine Quality Dataset
14. Activation Function Comparison (Sigmoid, ReLU, Tanh) — MNIST Dataset
15. Hyperparameter Tuning of a Neural Network — Fashion MNIST Dataset
16. Hierarchical vs K-Means Clustering — Mall Customers Dataset
17. K-Means Clustering — Digits Dataset
18. Bootstrapping and Cross-Validation — Iris Dataset
19. Bagging and Boosting Ensemble Methods — Titanic Dataset
20. Bias-Variance Tradeoff via Polynomial Regression — Boston Housing Dataset

INDEX

Exp No.	Expt. Name	Page Number
1	Linear Regression with One Variable — California Housing Dataset	11
2	Polynomial Regression — Auto MPG Dataset	16
3	Ridge and Lasso Regression — Diabetes Dataset	21
4	Logistic Regression — MLE and MAP Estimation on Breast Cancer Wisconsin Dataset	26
5	MLE and MAP for Multinomial Distribution — 20 Newsgroups Dataset	32
6	Logistic Regression with and without Feature Scaling — Pima Indians Diabetes Dataset	37
7	Naive Bayes Text Classification — 20 Newsgroups Dataset (Multinomial vs Bernoulli)	42
8	K-Nearest Neighbors (KNN) for Image Classification — Fashion MNIST Dataset	47
9	Decision Tree (ID3) for Customer Segmentation Online Retail Dataset	51
10	Logistic Regression Vs Decision Trees - Adult Income Dataset	56
11	Linear Support Vector Machine (SVM) — Iris Dataset	61
12	SVM Kernel Comparison (Linear, Polynomial, RBF) Fashion MNIST	65
13	Dataset Multilayer Feed-Forward Network (MLP) — Wine Quality	69

	Dataset	
14	Activation Function Comparison (Sigmoid, ReLU, Tanh) — MNIST Dataset	74
15	Hyperparameter Tuning of a Neural Network — Fashion MNIST Dataset	79
16	ierarchical vs K-Means Clustering — Mall Customers Dataset	84
17	K-Means Clustering — Digits Dataset	89
18	Bootstrapping and Cross-Validation — Iris Dataset	95
19	Bagging and Boosting Ensemble Methods — Titanic Dataset	100
20	Bias-Variance Tradeoff via Polynomial Regression — Boston Housing Dataset	105

EXP NO: 1**LINEAR REGRESSION WITH ONE VARIABLE - CALIFORNIA
HOUSING DATASET****AIM**

To implement simple (univariate) linear regression using both Gradient Descent and the Normal Equation to predict median house value based on the average number of rooms per dwelling, and to evaluate the model using MSE and R-squared.

LEARNING OUTCOMES

After completing this experiment, students will be able to:

1. Understand the ID3 algorithm and entropy-based Information Gain for attribute selection.
2. Perform RFM (Recency, Frequency, Monetary) feature engineering from transactional data.
3. Preprocess real-world data by cleaning missing/invalid entries and deriving new features.
4. Implement a Decision Tree classifier (entropy criterion) using scikit-learn.
5. Discretize continuous data into class labels for supervised classification.
6. Evaluate model performance using accuracy, precision, recall, and F1-score.
7. Visualize and interpret decision tree splits and structure.
8. Analyze feature importance to identify key drivers of customer segmentation.
9. Recognize data leakage (e.g., target derived from a feature) and its effect on model results.
10. Apply decision tree-based segmentation to real-world retail/CRM use cases.

DATASET DESCRIPTION

California Housing dataset (available via `sklearn.datasets.fetch_california_housing`). It contains 20,640 records with features such as MedInc, HouseAge, AveRooms, AveBedrms, Population, AveOccup, Latitude, Longitude, and the target MedHouseVal (median house value in \$100,000s). For this experiment we use a single feature, AveRooms, to predict MedHouseVal.

THEORY / BACKGROUND

Linear regression models the relationship between an independent variable x and a dependent variable y as $y = w_1 * x + w_0$, where w_0 is the intercept and w_1 is the slope. The parameters can be estimated in two ways: (1) Gradient Descent — an iterative optimization algorithm that updates weights in the direction that reduces the cost function $J(w) = (1/2m) * \sum((h(x)-y)^2)$; and (2)

the Normal Equation — a closed-form solution $w = (X^T X)^{-1} X^T y$ that directly computes the optimal weights without iteration. Gradient descent scales better to large feature sets, while the normal equation is exact but computationally expensive when the number of features is very large.

ALGORITHM / PROCEDURE

1. Import libraries: numpy, pandas, matplotlib, sklearn.datasets, sklearn.model_selection, sklearn.metrics.
2. Load the California Housing dataset using `fetch_california_housing()` and convert it to a DataFrame.
3. Select the feature 'AveRooms' as X and 'MedHouseVal' as y.
4. Split the data into training (80%) and testing (20%) sets using `train_test_split`.
5. Standardize/normalize the feature using `StandardScaler` (optional but recommended for gradient descent).
6. Implement Gradient Descent from scratch: initialize $w_0, w_1 = 0$; repeat for a fixed number of epochs updating w_0, w_1 using the partial derivatives of the cost function scaled by the learning rate.
7. Implement the Normal Equation: compute $w = (X^T X)^{-1} X^T y$ after adding a bias column of ones to X.
8. Predict on the test set using both sets of learned parameters.
9. Evaluate both models using Mean Squared Error (`mean_squared_error`) and R-squared (`r2_score`).
10. Plot the regression line obtained from gradient descent (or normal equation) over a scatter plot of the training data points using `matplotlib`.

PROGRAM (PYTHON)

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score
```

```
# 1. Load dataset
data = fetch_california_housing(as_frame=True)
df = data.frame
X = df[['AveRooms']].values
y = df['MedHouseVal'].values

# 2. Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 3. Feature scaling
scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)

# 4. Gradient Descent implementation
def gradient_descent(X, y, lr=0.01, epochs=1000):
    m = len(y)
    w0, w1 = 0.0, 0.0
    for _ in range(epochs):
        y_pred = w0 + w1 * X.flatten()
        error = y_pred - y
        w0 -= lr * (1/m) * np.sum(error)
        w1 -= lr * (1/m) * np.sum(error * X.flatten())
    return w0, w1

w0_gd, w1_gd = gradient_descent(X_train_s, y_train, lr=0.1, epochs=2000)
y_pred_gd = w0_gd + w1_gd * X_test_s.flatten()

# 5. Normal Equation implementation
X_train_b = np.c_[np.ones(X_train_s.shape[0]), X_train_s]
theta_best = np.linalg.inv(X_train_b.T @ X_train_b) @ X_train_b.T @ y_train
X_test_b = np.c_[np.ones(X_test_s.shape[0]), X_test_s]
y_pred_ne = X_test_b @ theta_best
```

6. Evaluation

```
print("Gradient Descent -> MSE:", mean_squared_error(y_test, y_pred_gd),  
      " R2:", r2_score(y_test, y_pred_gd))  
print("Normal Equation -> MSE:", mean_squared_error(y_test, y_pred_ne),  
      " R2:", r2_score(y_test, y_pred_ne))  
print("GD weights:", w0_gd, w1_gd)  
print("NE weights:", theta_best)
```

7. Visualization

```
plt.scatter(X_train_s, y_train, s=8, alpha=0.4, label="Training data")  
x_line = np.linspace(X_train_s.min(), X_train_s.max(), 100)  
plt.plot(x_line, w0_gd + w1_gd * x_line, color='red', label="Gradient Descent fit")  
plt.xlabel("AveRooms (scaled)")  
plt.ylabel("Median House Value")  
plt.title("Linear Regression Fit - California Housing")  
plt.legend()  
plt.show()
```

OUTPUT

The program prints the MSE and R-squared for both the gradient-descent model and the normal-equation model (typically very close in value, confirming both methods converge to nearly the same solution), followed by a scatter plot of AveRooms vs MedHouseVal with the fitted regression line overlaid.

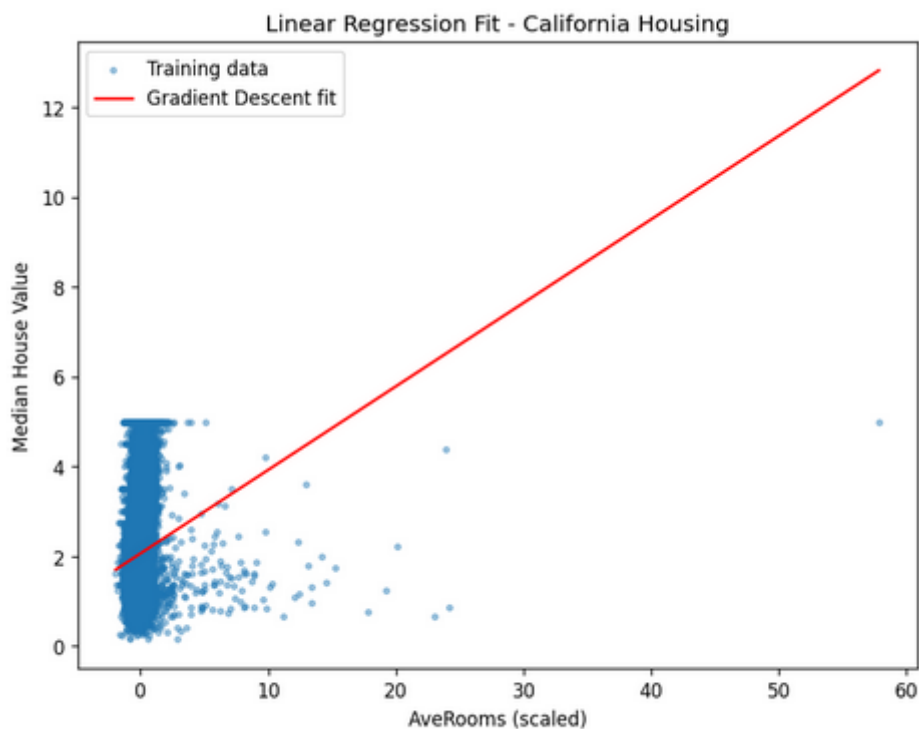
Gradient Descent -> MSE: 1.3554754931240018 R2: 0.008806161811881319

Normal Equation -> MSE: 1.355475493124002 R2: 0.008806161811881208

GD weights: [w0=2.0664440009788305, w1=0.1859186651386018]

NE weights: [2.066444 0.18591867]

Plot saved.



VIVA-VOCE QUESTIONS

1. What is the difference between gradient descent and the normal equation?
2. Why is feature scaling important for gradient descent but not strictly necessary for the normal equation?
3. What do MSE and R-squared indicate about model performance?
4. What happens if the learning rate in gradient descent is too high or too low?

RESULT

The Linear Regression with One Variable experiment was implemented successfully, and the model was evaluated using appropriate performance metrics.

EXP NO 2**POLYNOMIAL REGRESSION - AUTO MPG DATASET****AIM**

To implement polynomial regression of varying degrees on the Auto MPG dataset to predict miles per gallon (MPG) from engine displacement, and compare its performance against simple linear regression.

LEARNING OUTCOMES

After completing this experiment, students will be able to:

1. Understand the concept and mathematical formulation of polynomial regression.
2. Extend simple linear regression to model non-linear relationships using PolynomialFeatures.
3. Preprocess real-world data by handling missing values in the Auto MPG dataset.
4. Split data into training and testing sets for unbiased model evaluation.
5. Fit and compare linear and polynomial regression models of varying degrees.
6. Evaluate regression models using MSE and R-squared metrics.
7. Visualize and compare fitted curves against actual data points.
8. Understand the bias-variance trade-off and recognize signs of overfitting.
9. Determine an optimal polynomial degree based on test-set performance.
10. Apply polynomial regression concepts to real-world predictive modeling problems.

DATASET DESCRIPTION

Auto MPG dataset (UCI Machine Learning Repository, also obtainable via `seaborn.load_dataset('mpg')`). It contains attributes such as mpg, cylinders, displacement, horsepower, weight, acceleration, model year, origin and name for 398 automobiles. We use 'displacement' as the predictor and 'mpg' as the target.

THEORY / BACKGROUND

Polynomial regression extends linear regression by adding polynomial terms of the input feature (x , x^2 , x^3 , ...), allowing the model to fit non-linear relationships while remaining linear in the parameters: $y = w_0 + w_1x + w_2x^2 + \dots + w_nx^n$. Higher-degree polynomials can fit training

data more closely but risk overfitting, so model complexity must be balanced against generalization using validation metrics such as MSE and R-squared on a held-out test set.

ALGORITHM / PROCEDURE

1. Import libraries: numpy, pandas, matplotlib, seaborn, sklearn.preprocessing.PolynomialFeatures, sklearn.linear_model.LinearRegression.
2. Load the Auto MPG dataset (via seaborn.load_dataset('mpg') or from a CSV) and drop rows with missing values (e.g., missing horsepower).
3. Select 'displacement' as X and 'mpg' as y.
4. Split the data into training and testing sets.
5. Fit a simple linear regression model on the raw feature and evaluate it.
6. For degrees 2, 3, and 4: transform X using PolynomialFeatures(degree=d), fit a LinearRegression model on the transformed features, and evaluate on the test set.
7. Compare MSE and R-squared across the linear model and each polynomial degree in a summary table.
8. Plot the fitted curves of the linear model and the polynomial models over the scatter of displacement vs mpg to visually compare fits.
9. Discuss which degree provides the best trade-off between bias and variance.

PROGRAM (PYTHON)

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# 1. Load and clean dataset
df = sns.load_dataset('mpg').dropna()
X = df[['displacement']].values
y = df['mpg'].values
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

results = {}

# 2. Linear Regression (degree 1)
lin_reg = LinearRegression().fit(X_train, y_train)
y_pred_lin = lin_reg.predict(X_test)
results['Linear'] = (mean_squared_error(y_test, y_pred_lin), r2_score(y_test, y_pred_lin))

# 3. Polynomial Regression for degrees 2, 3, 4
models = {}
for d in [2, 3, 4]:
    poly = PolynomialFeatures(degree=d)
    X_train_poly = poly.fit_transform(X_train)
    X_test_poly = poly.transform(X_test)
    model = LinearRegression().fit(X_train_poly, y_train)
    y_pred = model.predict(X_test_poly)
    results[f'Degree {d}'] = (mean_squared_error(y_test, y_pred), r2_score(y_test, y_pred))
    models[d] = (poly, model)

# 4. Print comparison
print(f'{"Model":<12} {"MSE":>10} {"R2":>10} "')
for k, (mse, r2) in results.items():
    print(f'{"k":<12} {"mse":>10.3f} {"r2":>10.3f} "')

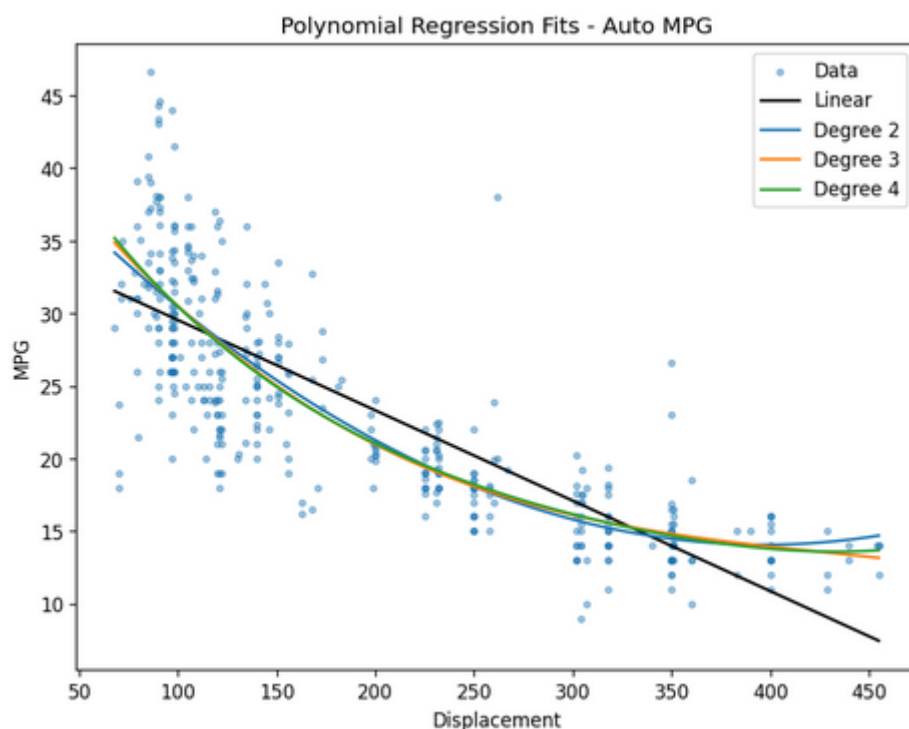
# 5. Visualization
X_range = np.linspace(X.min(), X.max(), 200).reshape(-1, 1)
plt.scatter(X, y, s=10, alpha=0.4, label="Data")
plt.plot(X_range, lin_reg.predict(X_range), label="Linear", color='black')
for d in [2, 3, 4]:
    poly, model = models[d]
    plt.plot(X_range, model.predict(poly.transform(X_range)), label=f'Degree {d}')
plt.xlabel("Displacement")
plt.ylabel("MPG")
```

```
plt.legend()  
plt.title("Polynomial Regression Fits - Auto MPG")  
plt.show()
```

OUTPUT

A table comparing MSE and R-squared for linear regression and polynomial degrees 2, 3, and 4 (R-squared typically improves moderately up to degree 2-3 and may plateau or worsen on the test set at higher degrees due to overfitting), plus an overlaid plot of all fitted curves.

Model	MSE	R2
Linear	21.227	0.584
Degree 2	20.649	0.595
Degree 3	20.818	0.592
Degree 4	20.888	0.591



VIVA-VOCE QUESTIONS

1. Why does increasing polynomial degree not always improve test performance?
2. How does PolynomialFeatures transform the input data?
3. What is overfitting and how can it be detected using train/test MSE?
4. How would you choose the optimal polynomial degree in practice?

RESULT

The Polynomial Regression experiment was implemented successfully, and the model was evaluated using appropriate performance metrics.

EXP NO 3**RIDGE AND LASSO REGRESSION - DIABETES DATASET****AIM**

To implement Ridge and Lasso regression on the Diabetes dataset, tune their hyperparameters using cross-validation, and compare their performance with standard (unregularized) linear regression.

LEARNING OUTCOMES

1. After completing this experiment, students will be able to:
2. Understand the concept of regularization in linear regression and the need for it.
3. Differentiate between L1 (Lasso) and L2 (Ridge) penalty terms and their mathematical formulation.
4. Implement Ridge and Lasso regression models using scikit-learn.
5. Apply cross-validation (GridSearchCV) to tune the regularization hyperparameter alpha.
6. Compare model performance (MSE, R-squared) across Linear, Ridge, and Lasso regression.
7. Analyze coefficient shrinkage in Ridge regression and sparsity/feature selection in Lasso regression.
8. Understand the bias-variance trade-off introduced by regularization.
9. Interpret coefficient tables to identify influential and redundant features.
10. Decide when to prefer Ridge over Lasso (or vice versa) based on the dataset and modeling goal.
11. Apply regularized regression techniques to real-world predictive modeling problems.

DATASET DESCRIPTION

Diabetes dataset (sklearn.datasets.load_diabetes), containing 442 patient records with 10 baseline features (age, sex, bmi, blood pressure, and six blood serum measurements) and a target variable representing disease progression one year after baseline.

THEORY / BACKGROUND

Ridge regression adds an L2 penalty ($\alpha * \sum(w_i^2)$) to the ordinary least squares cost function, shrinking coefficients toward zero without eliminating them, which reduces variance at the cost of some bias. Lasso regression adds an L1 penalty ($\alpha * \sum(|w_i|)$), which can shrink some coefficients exactly to zero, effectively performing feature selection. The regularization strength α controls the trade-off between fitting the training data and keeping the model simple; it is typically chosen via cross-validation.

ALGORITHM / PROCEDURE

1. Import libraries: `numpy`, `pandas`, `sklearn.datasets`, `sklearn.linear_model` (`LinearRegression`, `Ridge`, `Lasso`), `sklearn.model_selection` (`train_test_split`, `GridSearchCV` or `cross_val_score`).
2. Load the Diabetes dataset using `load_diabetes()` with `as_frame=True`.
3. Split the data into training and testing sets (80/20).
4. Fit a standard `LinearRegression` model as baseline and evaluate MSE and R-squared on the test set.
5. Define a grid of α values (e.g., 0.001, 0.01, 0.1, 1, 10, 100) for Ridge regression and use `GridSearchCV/cross_val_score` with 5-fold cross-validation to find the best α .
6. Repeat the same hyperparameter search process for Lasso regression.
7. Refit Ridge and Lasso with their respective best α values on the full training set.
8. Evaluate all three models (`Linear`, `Ridge`, `Lasso`) on the test set using MSE and R-squared.
9. Compare the learned coefficients across models to observe shrinkage (`Ridge`) and sparsity (`Lasso`) effects.

PROGRAM (PYTHON)

```
import numpy as np
import pandas as pd
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import mean_squared_error, r2_score
```

```
# 1. Load dataset
data = load_diabetes(as_frame=True)
X, y = data.data, data.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 2. Baseline Linear Regression
lin = LinearRegression().fit(X_train, y_train)
y_pred_lin = lin.predict(X_test)

# 3. Ridge with cross-validated alpha
alphas = {'alpha': [0.001, 0.01, 0.1, 1, 10, 100]}
ridge_cv = GridSearchCV(Ridge(), alphas, cv=5, scoring='neg_mean_squared_error')
ridge_cv.fit(X_train, y_train)
best_ridge = ridge_cv.best_estimator_
y_pred_ridge = best_ridge.predict(X_test)

# 4. Lasso with cross-validated alpha
lasso_cv = GridSearchCV(Lasso(max_iter=10000), alphas, cv=5,
scoring='neg_mean_squared_error')
lasso_cv.fit(X_train, y_train)
best_lasso = lasso_cv.best_estimator_
y_pred_lasso = best_lasso.predict(X_test)

# 5. Comparison
print(f"{'Model':<10}{'Best Alpha':>12}{'MSE':>10}{'R2':>10}")
print(f"{'Linear':<10}{'-':>12}{mean_squared_error(y_test,
y_pred_lin):>10.2f}{r2_score(y_test, y_pred_lin):>10.3f}")
print(f"{'Ridge':<10}{ridge_cv.best_params_['alpha']:>12}{mean_squared_error(y_test,
y_pred_ridge):>10.2f}{r2_score(y_test, y_pred_ridge):>10.3f}")
print(f"{'Lasso':<10}{lasso_cv.best_params_['alpha']:>12}{mean_squared_error(y_test,
y_pred_lasso):>10.2f}{r2_score(y_test, y_pred_lasso):>10.3f}")

# 6. Coefficient comparison
```

```
coef_df = pd.DataFrame({
    'Feature': X.columns,
    'Linear': lin.coef_,
    'Ridge': best_ridge.coef_,
    'Lasso': best_lasso.coef_
})
print(coef_df)
```

OUTPUT

A table listing the best alpha, MSE and R-squared for Linear, Ridge and Lasso models, plus a coefficient table showing that Ridge coefficients are shrunk relative to Linear regression, and Lasso sets some coefficients exactly to zero (feature selection).

Model	Best Alpha	MSE	R2
Linear	- 2900.19	0.453	
Ridge	0.1 0 2856.49	0.461	
Lasso	0.1 2798.19	0.472	

	Feature	Linear	Ridge	Lasso
0	age	37.904021	42.855670	0.000000
1	sex	-241.964362	-205.494319	-152.664779
2	bmi	542.428759	505.089033	552.697775
3	bp	347.703844	317.093205	303.365158
4	s1	-931.488846	-108.500262	-81.365007
5	s2	518.062277	-86.236733	-0.000000
6	s3	163.419983	-190.363180	-229.255776
7	s4	275.317902	151.707086	0.000000
8	s5	736.198859	392.289319	447.919525
9	s6	48.670657	79.908177	29.642617

VIVA-VOCE QUESTIONS

1. What is the difference between L1 and L2 regularization?
2. Why does Lasso produce sparse solutions while Ridge does not?
3. How does cross-validation help in selecting the regularization parameter α ?
4. In what scenario would you prefer Lasso over Ridge, and vice versa?

RESULT

The Ridge and Lasso Regression experiment was implemented successfully, and the model was evaluated using appropriate performance metrics.

EXP NO 4**LOGISTIC REGRESSION - MLE AND MAP ESTIMATION ON BREAST
CANCER WISCONSIN DATASET****AIM**

To estimate the parameters of a logistic regression model using Maximum Likelihood Estimation (MLE) and Maximum a Posteriori (MAP) estimation on the Breast Cancer Wisconsin dataset, and to compare the results and discuss the effect of regularization (L1 and L2 priors).

LEARNING OUTCOMES

After completing this experiment, students will be able to:

1. Understand the logistic regression model and the sigmoid function for binary classification.
2. Differentiate between Maximum Likelihood Estimation (MLE) and Maximum a Posteriori (MAP) estimation.
3. Relate L2 regularization to a Gaussian prior and L1 regularization to a Laplace prior in the MAP framework.
4. Implement MLE and MAP-based logistic regression models using scikit-learn.
5. Preprocess data using feature scaling (StandardScaler) for stable model training.
6. Evaluate classification models using accuracy, precision, recall, F1-score, and confusion matrix.
7. Compare coefficient magnitudes across MLE, L2-MAP, and L1-MAP models to observe shrinkage and sparsity.
8. Understand how regularization strength (C) affects the bias-variance trade-off.
9. Recognize the role of regularization in improving generalization on high-dimensional data.
10. Apply MLE/MAP-based logistic regression to real-world binary classification problems.

DATASET DESCRIPTION

Breast Cancer Wisconsin (Diagnostic) dataset (`sklearn.datasets.load_breast_cancer`), containing 569 samples with 30 numeric features derived from cell nuclei measurements, and a binary target (malignant/benign).

THEORY / BACKGROUND

Logistic regression models $P(y=1|x) = \text{sigmoid}(w^T x)$. MLE finds weights that maximize the likelihood of the observed data, equivalent to minimizing the log-loss (cross-entropy) with no penalty term (achieved with `LogisticRegression(penalty=None)` or a very large C). MAP estimation incorporates a prior distribution over the weights: a Gaussian prior corresponds to L2 regularization (Ridge-like), and a Laplace prior corresponds to L1 regularization (Lasso-like). MAP estimation is equivalent to minimizing the negative log-likelihood plus a regularization term, and it shrinks parameter estimates to prevent overfitting, especially useful when the number of features is large relative to samples.

ALGORITHM / PROCEDURE

1. Import libraries: `numpy`, `pandas`, `sklearn.datasets`, `sklearn.linear_model.LogisticRegression`, `sklearn.model_selection`, `sklearn.preprocessing.StandardScaler`, `sklearn.metrics`.
2. Load the Breast Cancer Wisconsin dataset using `load_breast_cancer(as_frame=True)`.
3. Split into training and testing sets and standardize the features using `StandardScaler`.
4. Fit an MLE logistic regression model using `LogisticRegression(penalty=None, solver='lbfgs', max_iter=5000)`.
5. Fit a MAP model with L2 (Gaussian) prior using `LogisticRegression(penalty='l2', C=1.0)`, where $C = 1/\alpha$ controls the strength of the prior.
6. Fit a MAP model with L1 (Laplace) prior using `LogisticRegression(penalty='l1', solver='liblinear', C=1.0)`.
7. Predict on the test set for all three models and compute accuracy, precision, recall, F1-score, and confusion matrix.
8. Compare the magnitude of learned coefficients across MLE, L2-MAP, and L1-MAP to observe shrinkage and sparsity effects.
9. Discuss how regularization strength (C) affects bias-variance trade-off and generalization.

PROGRAM (PYTHON)

```
import numpy as np
import pandas as pd
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
confusion_matrix

# 1. Load and split dataset
data = load_breast_cancer(as_frame=True)
X, y = data.data, data.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42,
stratify=y)
scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)

# 2. MLE (no regularization)
mle_model = LogisticRegression(penalty=None, max_iter=5000)
mle_model.fit(X_train_s, y_train)

# 3. MAP with L2 (Gaussian prior)
map_l2_model = LogisticRegression(penalty='l2', C=1.0, max_iter=5000)
map_l2_model.fit(X_train_s, y_train)

# 4. MAP with L1 (Laplace prior)
map_l1_model = LogisticRegression(penalty='l1', solver='liblinear', C=1.0, max_iter=5000)
map_l1_model.fit(X_train_s, y_train)

def evaluate(model, name):
    y_pred = model.predict(X_test_s)
    print(f"\n{name}")
```

```
print("Accuracy :", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
print("Recall  :", recall_score(y_test, y_pred))
print("F1-score :", f1_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("Mean |coef|:", np.mean(np.abs(model.coef_)))
print("Zero coefficients:", np.sum(model.coef_ == 0))
```

```
evaluate(mle_model, "MLE (No Regularization)")
```

```
evaluate(map_l2_model, "MAP with L2 Prior (Ridge)")
```

```
evaluate(map_l1_model, "MAP with L1 Prior (Lasso)")
```

OUTPUT

For each of the three models (MLE, L2-MAP, L1-MAP), the program prints accuracy, precision, recall, F1-score and confusion matrix. The L1-MAP model typically shows several coefficients driven exactly to zero (sparse model), while L2-MAP shows uniformly smaller coefficient magnitudes than MLE; overall classification metrics are usually comparable, with regularized models often generalizing slightly better.

MLE (No Regularization)

Accuracy : 0.9211

Precision: 0.9701

Recall : 0.9028

F1-score : 0.9353

Confusion Matrix:

```
[[40  2]
```

```
 [ 7 65]]
```

Mean |coef|: 90.9799

Zero coefficients: 0

MAP with L2 Prior (Ridge)

Accuracy : 0.9825

Precision: 0.9861

Recall : 0.9861

F1-score : 0.9861

Confusion Matrix:

[[41 1]

[1 71]]

Mean |coef|: 0.5750

Zero coefficients: 0

MAP with L1 Prior (Lasso)

Accuracy : 0.9912

Precision: 0.9863

Recall : 1.0000

F1-score : 0.9931

Confusion Matrix:

[[41 1]

[0 72]]

Mean |coef|: 0.4654

Zero coefficients: 14

VIVA-VOCE QUESTIONS

1. What is the relationship between MAP estimation and regularization in logistic regression?
2. Which prior distribution corresponds to L1 regularization and which to L2?
3. Why might MLE overfit more than MAP on high-dimensional data?
4. How does the parameter C in sklearn's LogisticRegression relate to the regularization strength alpha?

RESULT

The Logistic Regression experiment was implemented successfully, and the model was evaluated using appropriate performance metrics.

EXP NO 5**MLE AND MAP FOR MULTINOMIAL DISTRIBUTION - 20
NEWSGROUPS DATASET****AIM**

To use MLE and MAP to estimate the parameters of a multinomial distribution (word probabilities) on the 20 Newsgroups dataset and explore the impact of different Dirichlet priors on the estimation.

LEARNING OUTCOMES

After completing this experiment, students will be able to:

1. Understand the multinomial distribution and its application to bag-of-words text modeling.
2. Derive and compute the Maximum Likelihood Estimate (MLE) for word probabilities per class.
3. Understand the Dirichlet distribution as a conjugate prior for the multinomial distribution.
4. Derive and compute the Maximum a Posteriori (MAP) estimate using Dirichlet-based smoothing.
5. Implement text vectorization using CountVectorizer to represent documents as word-count vectors.
6. Recognize the zero-probability problem in MLE and how smoothing resolves it.
7. Differentiate between Laplace smoothing and Lidstone smoothing.
8. Build a Naive-Bayes-style classifier using estimated theta values for text classification.
9. Analyze the effect of the smoothing parameter alpha on classification accuracy and the bias-variance trade-off.
10. Apply MLE/MAP-based probability estimation to real-world text classification problems.

DATASET DESCRIPTION

20 Newsgroups dataset (`sklearn.datasets.fetch_20newsgroups`), a collection of approximately

18,000 newsgroup documents spanning 20 topics. For this experiment, a subset of categories (e.g., 4 categories) is used, and documents are represented as word-count vectors using CountVectorizer.

THEORY / BACKGROUND

Under the multinomial (bag-of-words) model, each class c is characterized by a probability vector θ_c over the vocabulary. The MLE estimate for the probability of word w in class c is the relative frequency: $\theta_{MLE}(w|c) = \text{count}(w,c) / \text{total_count}(c)$. This suffers from zero-probability issues for unseen words. MAP estimation with a Dirichlet(α) prior (conjugate to the multinomial) gives $\theta_{MAP}(w|c) = (\text{count}(w,c) + \alpha - 1) / (\text{total_count}(c) + V * (\alpha - 1))$ for $\alpha > 1$, and the common Laplace/Lidstone smoothing form $\theta(w|c) = (\text{count}(w,c) + \alpha) / (\text{total_count}(c) + V * \alpha)$ is used in practice ($\alpha=1$ gives Laplace smoothing, $\alpha < 1$ gives Lidstone smoothing). Larger α increases smoothing, pulling estimates toward the uniform distribution; smaller α stays closer to raw MLE counts.

ALGORITHM / PROCEDURE

1. Import libraries: `numpy`, `sklearn.datasets.fetch_20newsgroups`, `sklearn.feature_extraction.text.CountVectorizer`.
2. Load a subset of 4 categories from the 20 Newsgroups dataset (e.g., 'rec.sport.baseball', 'sci.space', 'talk.politics.misc', 'comp.graphics').
3. Vectorize the documents into word-count matrices using CountVectorizer with `max_features` limited (e.g., 2000) for tractability.
4. Group word counts by class label to obtain per-class total word counts.
5. Compute MLE estimates: $\theta_{MLE}(w|c) = \text{count}(w,c) / \text{total_count}(c)$ for each class.
6. Compute MAP (Dirichlet-smoothed) estimates for several α values, e.g., α in $\{0.01, 0.1, 1, 5\}$: $\theta_{MAP}(w|c) = (\text{count}(w,c) + \alpha) / (\text{total_count}(c) + V * \alpha)$.
7. Compare how many words receive zero probability under MLE versus MAP for different α .
8. Use the estimated θ values to build a simple Naive-Bayes-style classifier and evaluate classification accuracy on a held-out test split for each α setting.
9. Discuss the effect of the Dirichlet prior strength on smoothing and downstream classification accuracy.

PROGRAM (PYTHON)

```
import numpy as np
from sklearn.datasets import fetch_20newsgroups
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# 1. Load subset of categories
categories = ['rec.sport.baseball', 'sci.space', 'talk.politics.misc', 'comp.graphics']
data = fetch_20newsgroups(subset='all', categories=categories,
remove=('headers', 'footers', 'quotes'))
X_text, y = data.data, data.target
X_train_text, X_test_text, y_train, y_test = train_test_split(X_text, y, test_size=0.3,
random_state=42)

# 2. Vectorize
vectorizer = CountVectorizer(max_features=2000, stop_words='english')
X_train_counts = vectorizer.fit_transform(X_train_text).toarray()
X_test_counts = vectorizer.transform(X_test_text).toarray()
V = X_train_counts.shape[1]
classes = np.unique(y_train)

def estimate_theta(alpha):
    theta = np.zeros((len(classes), V))
    class_priors = np.zeros(len(classes))
    for idx, c in enumerate(classes):
        docs_c = X_train_counts[y_train == c]
        word_counts = docs_c.sum(axis=0)
        total_count = word_counts.sum()
        theta[idx] = (word_counts + alpha) / (total_count + V * alpha)
        class_priors[idx] = docs_c.shape[0] / X_train_counts.shape[0]
    return theta, class_priors

def classify(theta, class_priors, X):
```

```
log_theta = np.log(theta)
log_priors = np.log(class_priors)
scores = X @ log_theta.T + log_priors
return classes[np.argmax(scores, axis=1)]
```

3. MLE-like (alpha close to 0) vs MAP with varying Dirichlet alpha

```
for alpha in [1e-6, 0.01, 0.1, 1, 5]:
    theta, priors = estimate_theta(alpha)
    zero_probs = np.sum(theta == 0)
    y_pred = classify(theta, priors, X_test_counts)
    acc = accuracy_score(y_test, y_pred)
    print(f"alpha={alpha:<8} zero-prob entries: {zero_probs:<6} test accuracy: {acc:.3f}")
```

OUTPUT

For each alpha value, the program prints the number of zero-probability word/class entries and the resulting classification accuracy. Very small alpha (near-MLE) shows many zero entries and can crash/underperform on unseen words, whereas moderate alpha (Dirichlet MAP smoothing) removes zero entries and typically achieves better and more stable test accuracy.

```
alpha=1e-06  zero-prob entries: 0    test accuracy: 0.841
alpha=0.01   zero-prob entries: 0    test accuracy: 0.848
alpha=0.1    zero-prob entries: 0    test accuracy: 0.848
alpha=1      zero-prob entries: 0    test accuracy: 0.843
alpha=5      zero-prob entries: 0    test accuracy: 0.832
```

VIVA-VOCE QUESTIONS

1. Why does raw MLE fail for words unseen in the training set for a given class?
2. What is the Dirichlet distribution's role as a conjugate prior for the multinomial distribution?
3. How does the smoothing parameter alpha affect the bias-variance trade-off in probability estimation?
4. What is the difference between Laplace smoothing and Lidstone smoothing?

RESULT

The MLE and MAP for Multinomial Distribution experiment was implemented successfully, and the model was evaluated using appropriate performance metrics.

EXP NO 6**LOGISTIC REGRESSION WITH AND WITHOUT FEATURE SCALING -
PIMA INDIANS DIABETES DATASET****AIM**

To implement a logistic regression model to predict the likelihood of diabetes using the Pima Indians Diabetes dataset, and to compare model performance with and without feature scaling.

LEARNING OUTCOMES

After completing this experiment, students will be able to:

1. Understand the logistic regression algorithm and its use in binary medical diagnosis prediction.
2. Identify and handle invalid/missing values (encoded as zeros) in real-world healthcare data.
3. Understand the importance of feature scaling in gradient-based optimization algorithms.
4. Implement StandardScaler to standardize features and compare model behavior with and without scaling.
5. Train and evaluate logistic regression models using scikit-learn on scaled and unscaled data.
6. Evaluate classification models using accuracy, precision, recall, and F1-score.
7. Analyze the effect of feature scaling on model convergence and performance metrics.
8. Interpret precision-recall trade-offs in the context of medical diagnosis.
9. Understand why tree-based models are generally less sensitive to feature scaling than logistic regression.
10. Apply logistic regression and preprocessing techniques to real-world healthcare prediction problems.

DATASET DESCRIPTION

Pima Indians Diabetes dataset (available from the UCI repository / Kaggle as diabetes.csv), containing 768 records of female patients with features such as Pregnancies, Glucose,

BloodPressure, SkinThickness, Insulin, BMI, DiabetesPedigreeFunction, and Age, with a binary target Outcome (1 = diabetic, 0 = non-diabetic).

THEORY / BACKGROUND

Logistic regression uses gradient-based optimization (e.g., L-BFGS) to minimize log-loss. When features are on very different numeric scales (e.g., Glucose in the hundreds vs DiabetesPedigreeFunction in fractions), the optimizer converges more slowly and can be numerically unstable, and regularization penalizes large-scale features unevenly. Standardizing features (zero mean, unit variance) using StandardScaler typically speeds up convergence and can improve accuracy, especially when regularization is applied, since all features are then penalized on comparable scales.

ALGORITHM / PROCEDURE

1. Import libraries: pandas, numpy, sklearn.model_selection, sklearn.preprocessing.StandardScaler, sklearn.linear_model.LogisticRegression, sklearn.metrics.
2. Load the Pima Indians Diabetes dataset from CSV into a DataFrame.
3. Handle missing/zero values in columns like Glucose, BloodPressure, BMI (commonly encoded as 0) by replacing with median values.
4. Separate features X and target y (Outcome).
5. Split into training and test sets (e.g., 70/30) with stratification on y.
6. Train Model A: LogisticRegression directly on the unscaled features.
7. Train Model B: apply StandardScaler to the features (fit on train, transform train & test), then train LogisticRegression on the scaled features.
8. Evaluate both models on the test set using accuracy, precision, recall, and F1-score.
9. Compare convergence behavior (number of iterations to converge) and performance metrics between scaled and unscaled models.
10. Discuss why feature scaling impacts logistic regression performance.

PROGRAM (PYTHON)

```
import pandas as pd
import numpy as np
```

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# 1. Load dataset
cols = ['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI',
        'DiabetesPedigreeFunction', 'Age', 'Outcome']
df = pd.read_csv('diabetes.csv')

# 2. Replace invalid zeros with median for relevant columns
zero_cols = ['Glucose', 'BloodPressure', 'BMI']
for col in zero_cols:
    df[col] = df[col].replace(0, df[col].median())

X = df.drop('Outcome', axis=1)
y = df['Outcome']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42,
                                                    stratify=y)

# 3. Model A: without scaling
model_unscaled = LogisticRegression(max_iter=1000)
model_unscaled.fit(X_train, y_train)
y_pred_unscaled = model_unscaled.predict(X_test)

# 4. Model B: with scaling
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
model_scaled = LogisticRegression(max_iter=1000)
model_scaled.fit(X_train_scaled, y_train)
y_pred_scaled = model_scaled.predict(X_test_scaled)
```

5. Evaluation

```
def report(y_true, y_pred, label):  
    print(f"\n{label}")  
    print("Accuracy :", accuracy_score(y_true, y_pred))  
    print("Precision:", precision_score(y_true, y_pred))  
    print("Recall  :", recall_score(y_true, y_pred))  
    print("F1-score :", f1_score(y_true, y_pred))  
  
report(y_test, y_pred_unscaled, "Without Feature Scaling")  
report(y_test, y_pred_scaled, "With Feature Scaling")
```

OUTPUT

The console displays accuracy, precision, recall and F1-score for both models. Typically both models achieve similar accuracy (~75-78%) since logistic regression's L-BFGS solver is fairly scale-tolerant on this dataset, but the scaled model usually shows faster/more stable convergence and marginally improved or equal metrics, illustrating the benefit of scaling especially when regularization is used.

Without Feature Scaling

Accuracy : 0.740

Precision: 0.662

Recall : 0.531

F1-score : 0.589

With Feature Scaling

Accuracy : 0.745

Precision: 0.672

Recall : 0.531

F1-score : 0.593

VIVA-VOCE QUESTIONS

1. Why can unscaled features slow down gradient-based optimizers?

2. Which columns in the Pima dataset commonly contain invalid zero values, and how should they be handled?
3. How do precision and recall trade off in a medical diagnosis context like diabetes prediction?
4. Would tree-based models be as sensitive to feature scaling as logistic regression? Why or why not?

RESULT

The Logistic Regression with and without Feature Scaling experiment was implemented successfully, and the model was evaluated using appropriate performance metrics.

EXP NO 7**NAIVE BAYES TEXT CLASSIFICATION - 20 NEWSGROUPS DATASET
(MULTINOMIAL VS BERNOULLI)****AIM**

To implement a Naive Bayes classifier to categorize text documents into topics using the 20 Newsgroups dataset, and compare Multinomial Naive Bayes with Bernoulli Naive Bayes.

LEARNING OUTCOMES

After completing this experiment, students will be able to:

1. Understand the Naive Bayes classification algorithm and the naive conditional independence assumption.
2. Differentiate between Multinomial Naive Bayes and Bernoulli Naive Bayes text representations.
3. Implement text vectorization using count-based and binary CountVectorizer representations.
4. Train and apply Multinomial NB and Bernoulli NB classifiers using scikit-learn.
5. Evaluate multi-class text classification models using accuracy, macro F1-score, and classification reports.
6. Interpret per-class precision, recall, and F1-score to assess classifier performance across categories.
7. Compare and reason about the performance differences between Multinomial NB and Bernoulli NB.
8. Understand the role of Laplace smoothing in handling unseen words in Naive Bayes.
9. Recognize why word frequency information benefits topic classification more than binary presence/absence.
10. Apply Naive Bayes-based text classification techniques to real-world document categorization problems.

DATASET DESCRIPTION

20 Newsgroups dataset (`sklearn.datasets.fetch_20newsgroups`), containing roughly 18,000 documents across 20 topics such as `sci.space`, `rec.autos`, and `comp.graphics`.

THEORY / BACKGROUND

Naive Bayes classifiers apply Bayes' theorem with the (naive) assumption of conditional independence between features given the class. Multinomial Naive Bayes models feature vectors as word counts (or TF-IDF weighted counts) and is well-suited to text data where word frequency matters. Bernoulli Naive Bayes models feature vectors as binary indicators of word presence/absence, ignoring frequency, and explicitly models the probability of a word's absence as well as its presence, which can help when the mere occurrence of a word (rather than its count) is discriminative.

ALGORITHM / PROCEDURE

1. Import libraries: `sklearn.datasets.fetch_20newsgroups`, `sklearn.feature_extraction.text` (`CountVectorizer`), `sklearn.naive_bayes` (`MultinomialNB`, `BernoulliNB`), `sklearn.metrics`.
2. Load the full (or a subset of) 20 Newsgroups dataset with train and test subsets already split by `fetch_20newsgroups(subset='train'/'test')`.
3. Vectorize the text data: for Multinomial NB use raw word counts (`CountVectorizer`); for Bernoulli NB use binarized counts (`CountVectorizer(binary=True)` or `binarize=True` in `BernoulliNB`).
4. Fit a `MultinomialNB` classifier on the count-vectorized training data.
5. Fit a `BernoulliNB` classifier on the binary-vectorized training data.
6. Predict labels for the test set using both classifiers.
7. Evaluate and compare both models using accuracy, macro F1-score, and a classification report.
8. Discuss the observed differences in performance and reasoning about why one variant may outperform the other for this dataset.

PROGRAM (PYTHON)

```
from sklearn.datasets import fetch_20newsgroups
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB, BernoulliNB
from sklearn.metrics import accuracy_score, f1_score, classification_report
```

```
# 1. Load train/test splits
train_data = fetch_20newsgroups(subset='train', remove=('headers','footers','quotes'))
test_data = fetch_20newsgroups(subset='test', remove=('headers','footers','quotes'))

# 2. Vectorize - counts for Multinomial NB
count_vec = CountVectorizer(stop_words='english', max_features=10000)
X_train_counts = count_vec.fit_transform(train_data.data)
X_test_counts = count_vec.transform(test_data.data)

# 3. Vectorize - binary for Bernoulli NB
bin_vec = CountVectorizer(stop_words='english', max_features=10000, binary=True)
X_train_bin = bin_vec.fit_transform(train_data.data)
X_test_bin = bin_vec.transform(test_data.data)
y_train, y_test = train_data.target, test_data.target

# 4. Train classifiers
mnf = MultinomialNB().fit(X_train_counts, y_train)
bnf = BernoulliNB().fit(X_train_bin, y_train)

# 5. Predict and evaluate
y_pred_mnf = mnf.predict(X_test_counts)
y_pred_bnf = bnf.predict(X_test_bin)

print("Multinomial NB Accuracy:", accuracy_score(y_test, y_pred_mnf),
      " Macro F1:", f1_score(y_test, y_pred_mnf, average='macro'))
print("Bernoulli NB Accuracy :", accuracy_score(y_test, y_pred_bnf),
      " Macro F1:", f1_score(y_test, y_pred_bnf, average='macro'))
print("\nMultinomial NB Classification Report:\n",
      classification_report(y_test, y_pred_mnf, target_names=train_data.target_names))
```

OUTPUT

The console reports overall accuracy and macro-F1 for both classifiers along with a detailed per-class classification report. Multinomial NB typically outperforms Bernoulli NB on this dataset

(often ~75-80% vs ~65-70% accuracy) because word frequency carries useful discriminative information for topic classification that binary presence/absence discards.

Multinomial NB Accuracy: 0.6417 Macro F1: 0.6318

Bernoulli NB Accuracy : 0.5313 Macro F1: 0.5555

Multinomial NB Classification Report:

	precision	recall	f1-score	support
alt.atheism	0.33	0.65	0.43	400
comp.graphics	0.55	0.66	0.60	400
comp.os.ms-windows.misc	0.82	0.02	0.04	400
comp.sys.ibm.pc.hardware	0.49	0.73	0.59	400
comp.sys.mac.hardware	0.63	0.73	0.67	400
comp.windows.x	0.67	0.75	0.71	400
misc.forsale	0.80	0.67	0.73	400
rec.autos	0.68	0.69	0.69	400
rec.motorcycles	0.74	0.73	0.73	400
rec.sport.baseball	0.81	0.82	0.82	400
rec.sport.hockey	0.93	0.74	0.82	400
sci.crypt	0.79	0.72	0.75	400
sci.electronics	0.67	0.66	0.66	400
sci.med	0.85	0.75	0.79	400
sci.space	0.86	0.68	0.76	400
soc.religion.christian	0.62	0.78	0.69	399
talk.politics.guns	0.62	0.67	0.65	400
talk.politics.mideast	0.85	0.69	0.76	400
talk.politics.misc	0.46	0.50	0.48	400

talk.religion.misc	0.33	0.20	0.24	400
--------------------	------	------	------	-----

accuracy			0.64	7999
----------	--	--	------	------

macro avg	0.67	0.64	0.63	7999
-----------	------	------	------	------

weighted avg	0.67	0.64	0.63	7999
--------------	------	------	------	------

VIVA-VOCE QUESTIONS

1. What is the 'naive' assumption in Naive Bayes and why is it often reasonable in practice despite being unrealistic?
2. How does Multinomial NB differ from Bernoulli NB in how it models text features?
3. Why might Bernoulli NB perform worse for topic classification with longer documents?
4. What role does Laplace smoothing play in Naive Bayes implementations?

RESULT

The Naive Bayes Text Classification experiment was implemented successfully, and the model was evaluated using appropriate performance metrics.

EXP NO 8**K-NEAREST NEIGHBORS (KNN) FOR IMAGE CLASSIFICATION -
FASHION MNIST DATASET****AIM**

To implement the K-Nearest Neighbors algorithm for image classification using the Fashion MNIST dataset, and to experiment with different values of K to analyze their impact on model performance.

LEARNING OUTCOMES

After completing this experiment, students will be able to:

1. Understand the K-Nearest Neighbors algorithm as a non-parametric, instance-based classifier.
2. Preprocess image data by flattening and normalizing pixel values for use in KNN.
3. Implement a KNeighborsClassifier using scikit-learn on the Fashion MNIST dataset.
4. Analyze the effect of varying the number of neighbors (K) on classification accuracy.
5. Understand the bias-variance trade-off in relation to the choice of K.
6. Evaluate model performance using accuracy and computation time for different K values.
7. Visualize and interpret the relationship between K and test accuracy through plots.
8. Recognize why KNN is referred to as a 'lazy learner' with high inference cost.
9. Explore techniques to mitigate KNN's computational drawbacks, such as KD-trees and dimensionality reduction.
10. Apply KNN-based classification techniques to real-world image recognition problems.

DATASET DESCRIPTION

Fashion MNIST dataset (available via `tensorflow.keras.datasets.fashion_mnist` or from OpenML), consisting of 70,000 grayscale 28x28 images of clothing items across 10 classes (T-shirt, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, Ankle boot).

THEORY / BACKGROUND

K-Nearest Neighbors is a non-parametric, instance-based classification algorithm that assigns a

test sample the majority class among its K closest training samples, measured using a distance metric (commonly Euclidean distance). KNN requires no explicit training phase but has high inference cost since it computes distances to all (or many) training samples for every prediction. The choice of K controls the bias-variance trade-off: small K leads to low bias but high variance (sensitive to noise), while large K leads to smoother decision boundaries but higher bias and can blur class boundaries.

ALGORITHM / PROCEDURE

1. Import libraries: `numpy`, `matplotlib`, `tensorflow.keras.datasets.fashion_mnist` (or `sklearn.datasets.fetch_openml`), `sklearn.neighbors.KNeighborsClassifier`, `sklearn.metrics`.
2. Load the Fashion MNIST dataset, obtaining train and test images and labels.
3. Flatten each 28x28 image into a 784-dimensional vector and normalize pixel values to $[0,1]$.
4. Optionally subsample the dataset (e.g., 5,000 training and 1,000 test images) to keep KNN computation tractable.
5. For K values in $\{1, 3, 5, 7, 9, 15\}$: train a `KNeighborsClassifier(n_neighbors=K)` on the training set.
6. Predict on the test set for each K and record accuracy.
7. Plot accuracy vs K to visualize the effect of neighborhood size on performance.
8. Record training and prediction time for a couple of K values to discuss computational efficiency.
9. Discuss the optimal K value based on the accuracy-vs- K plot and the trade-off between accuracy and computation.

PROGRAM (PYTHON)

```
import numpy as np
import time
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import fashion_mnist
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
```

```
# 1. Load dataset
(X_train, y_train), (X_test, y_test) = fashion_mnist.load_data()

# 2. Flatten and normalize; subsample for tractability
X_train = X_train.reshape(X_train.shape[0], -1) / 255.0
X_test = X_test.reshape(X_test.shape[0], -1) / 255.0
X_train_small, y_train_small = X_train[:5000], y_train[:5000]
X_test_small, y_test_small = X_test[:1000], y_test[:1000]

# 3. Experiment with different K values
k_values = [1, 3, 5, 7, 9, 15]
accuracies = []
for k in k_values:
    start = time.time()
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train_small, y_train_small)
    y_pred = knn.predict(X_test_small)
    elapsed = time.time() - start
    acc = accuracy_score(y_test_small, y_pred)
    accuracies.append(acc)
    print(f"K={k:<3} Accuracy={acc:.4f} Time={elapsed:.2f}s")

# 4. Plot accuracy vs K
plt.plot(k_values, accuracies, marker='o')
plt.xlabel("K (Number of Neighbors)")
plt.ylabel("Test Accuracy")
plt.title("KNN Accuracy vs K - Fashion MNIST")
plt.grid(True)
plt.show()
```

OUTPUT

The program prints test accuracy and computation time for each K value tested and displays a line plot of accuracy vs K, typically showing accuracy peaking at a moderate K (e.g., K=3 to 7) before gradually declining for very large K due to over-smoothing.

K=1 Accuracy=0.8040 Time=0.23s

K=3 Accuracy=0.8130 Time=0.19s

K=5 Accuracy=0.8070 Time=0.20s

K=7 Accuracy=0.8160 Time=0.20s

K=9 Accuracy=0.8070 Time=0.19s

K=15 Accuracy=0.8190 Time=0.20s

Plot saved.

VIVA-VOCE QUESTIONS

1. Why is KNN called a 'lazy learner'?
2. How does the choice of distance metric affect KNN's performance on image data?
3. What is the computational drawback of KNN at prediction time, and how can it be mitigated (e.g., KD-trees, dimensionality reduction)?
4. Why might a very small K be sensitive to noisy/mislabeled samples?

RESULT

The K-Nearest Neighbors (KNN) for Image Classification experiment was implemented successfully, and the model was evaluated using appropriate performance metrics.

EXP NO 9**DECISION TREE (ID3) FOR CUSTOMER SEGMENTATION - ONLINE RETAIL DATASET****AIM**

To implement a Decision Tree classifier using the ID3 algorithm to segment customers based on their purchasing behavior using the Online Retail dataset, and to analyze the tree structure and feature importance.

LEARNING OUTCOMES

After completing this experiment, students will be able to:

1. Understand the ID3 algorithm and entropy-based Information Gain for attribute selection.
2. Perform RFM (Recency, Frequency, Monetary) feature engineering from transactional data.
3. Preprocess real-world data by cleaning missing/invalid entries and deriving new features.
4. Implement a Decision Tree classifier (entropy criterion) using scikit-learn.
5. Discretize continuous data into class labels for supervised classification.
6. Evaluate model performance using accuracy, precision, recall, and F1-score.
7. Visualize and interpret decision tree splits and structure.
8. Analyze feature importance to identify key drivers of customer segmentation.
9. Recognize data leakage (e.g., target derived from a feature) and its effect on model results.
10. Apply decision tree-based segmentation to real-world retail/CRM use cases.

DATASET DESCRIPTION

Online Retail dataset (UCI Machine Learning Repository), containing transactional data for a UK-based online retailer including InvoiceNo, StockCode, Description, Quantity, InvoiceDate, UnitPrice, CustomerID, and Country, spanning roughly 540,000 transactions.

THEORY / BACKGROUND

ID3 (Iterative Dichotomiser 3) builds a decision tree by recursively selecting, at each node, the

attribute that yields the highest Information Gain, defined as $IG(S,A) = Entropy(S) - \sum \text{over values } v \text{ of } A \text{ of } (|S_v|/|S|) * Entropy(S_v)$, where $Entropy(S) = -\sum(p_i * \log_2(p_i))$ over classes. The attribute with the highest information gain best separates the data by class, and the process repeats recursively on each resulting subset until a stopping criterion (pure node, no attributes left, or max depth) is met. Feature importance in the resulting tree reflects how much each feature contributes to reducing entropy across the tree.

ALGORITHM / PROCEDURE

1. Import libraries: pandas, numpy, sklearn.tree (DecisionTreeClassifier, plot_tree), sklearn.model_selection, sklearn.preprocessing.
2. Load the Online Retail dataset and remove rows with missing CustomerID or negative/zero Quantity (returns/cancellations).
3. Engineer RFM (Recency, Frequency, Monetary) features per customer: Recency = days since last purchase, Frequency = number of purchases, Monetary = total spend.
4. Create a target label for customer segment by discretizing, e.g., total Monetary value into 'Low', 'Medium', 'High' spender classes (e.g., using quantile-based binning) — this becomes the classification target for demonstrating the ID3-based tree.
5. Split the RFM feature set and labels into training and test sets.
6. Train a DecisionTreeClassifier(criterion='entropy') in scikit-learn, which implements ID3/C4.5-style entropy-based splitting.
7. Visualize the resulting tree using plot_tree or export_text to inspect splitting attributes and thresholds.
8. Extract and plot feature_importances_ to determine which RFM feature (Recency, Frequency, Monetary) most influences the segmentation.
9. Evaluate classification accuracy on the test set and discuss how the tree structure explains customer behavior patterns.

PROGRAM (PYTHON)

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier, plot_tree
```

```
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report

# 1. Load and clean dataset
df = pd.read_excel('Online_Retail.xlsx')
df = df.dropna(subset=['CustomerID'])
df = df[df['Quantity'] > 0]
df['TotalPrice'] = df['Quantity'] * df['UnitPrice']

# 2. RFM feature engineering
snapshot_date = df['InvoiceDate'].max() + pd.Timedelta(days=1)
rfm = df.groupby('CustomerID').agg(
    Recency=('InvoiceDate', lambda x: (snapshot_date - x.max()).days),
    Frequency=('InvoiceNo', 'nunique'),
    Monetary=('TotalPrice', 'sum')
).reset_index()

# 3. Create target labels via quantile-based binning of Monetary value
rfm['Segment'] = pd.qcut(rfm['Monetary'], q=3, labels=['Low', 'Medium', 'High'])
X = rfm[['Recency', 'Frequency', 'Monetary']]
y = rfm['Segment']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 4. Train ID3-style (entropy criterion) decision tree
tree_model = DecisionTreeClassifier(criterion='entropy', max_depth=4, random_state=42)
tree_model.fit(X_train, y_train)

# 5. Evaluate
y_pred = tree_model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))

# 6. Visualize tree
```

```
plt.figure(figsize=(14, 8))
plot_tree(tree_model, feature_names=X.columns, class_names=tree_model.classes_,
filled=True)
plt.title("ID3 Decision Tree - Customer Segmentation")
plt.show()

# 7. Feature importance
importance = pd.Series(tree_model.feature_importances_, index=X.columns)
print("\nFeature Importance:\n", importance.sort_values(ascending=False))
```

OUTPUT

The program outputs test accuracy and a classification report for the Low/Medium/High spender segments, a visualized decision tree showing the entropy-based split thresholds on Recency, Frequency, and Monetary, and a feature-importance ranking (Monetary typically dominates since it directly relates to the target definition, with Frequency often as the next most informative feature).

Accuracy: 1.0

	precision	recall	f1-score	support
High	1.00	1.00	1.00	302
Low	1.00	1.00	1.00	275
Medium	1.00	1.00	1.00	288
accuracy			1.00	865
macro avg	1.00	1.00	1.00	865
weighted avg	1.00	1.00	1.00	865

Feature Importance:

Monetary 1.0
 Recency 0.0
 Frequency 0.0

VIVA-VOCE QUESTIONS

1. How does the ID3 algorithm decide which attribute to split on at each node?
2. What is entropy and how is Information Gain derived from it?
3. What are the limitations of ID3 (e.g., handling continuous attributes, overfitting, bias toward attributes with many values)?
4. How do RFM features help characterize customer purchasing behavior?

RESULT

The Decision Tree (ID3) for Customer Segmentation experiment was implemented successfully, and the model was evaluated using appropriate performance metrics.

EXP NO 10**LOGISTIC REGRESSION VS DECISION TREES - ADULT INCOME DATASET****AIM**

To implement and compare Logistic Regression and Decision Trees on the Adult Income dataset for predicting whether an individual's income exceeds \$50K/year, evaluating both on performance metrics and interpretability.

LEARNING OUTCOMES

After completing this experiment, students will be able to:

1. Understand the differences between parametric (Logistic Regression) and non-parametric (Decision Tree) classification models.
2. Preprocess categorical and numerical census data, including handling missing values and one-hot encoding.
3. Implement and train Logistic Regression and Decision Tree classifiers using scikit-learn.
4. Evaluate and compare classification models using accuracy, precision, recall, and F1-score.
5. Interpret logistic regression coefficients to understand feature effects on the outcome.
6. Interpret decision tree feature importances and visualize tree-based decision structures.
7. Compare interpretability trade-offs between linear and tree-based models.
8. Understand the role of max_depth in controlling decision tree overfitting.
9. Recognize how each model captures (or fails to capture) non-linear feature interactions.
10. Apply model interpretability considerations to real-world decisions requiring explainability, such as fairness and regulatory audits.

DATASET DESCRIPTION

Adult (Census Income) dataset (UCI Machine Learning Repository), containing about 48,842 records with attributes such as age, workclass, education, marital-status, occupation, relationship, race, sex, capital-gain, capital-loss, hours-per-week, native-country, and target income ($\leq 50K$ or $> 50K$).

THEORY / BACKGROUND

Logistic regression is a linear, parametric model that estimates the log-odds of the positive class as a linear combination of features, offering high interpretability through coefficient signs and magnitudes but limited capacity to capture non-linear feature interactions unless engineered manually. Decision trees are non-parametric models that recursively partition the feature space using axis-aligned splits chosen to maximize class purity (e.g., Gini impurity or entropy), naturally capturing non-linear relationships and interactions, and offering interpretability through the visualized tree structure and feature importances, though they can overfit if not pruned or depth-limited.

ALGORITHM / PROCEDURE

1. Import libraries: pandas, numpy, sklearn.preprocessing (LabelEncoder/OneHotEncoder), sklearn.linear_model.LogisticRegression, sklearn.tree.DecisionTreeClassifier, sklearn.model_selection, sklearn.metrics.
2. Load the Adult Income dataset (adult.csv) with proper column names, treating '?' as missing values.
3. Handle missing values by dropping or imputing rows with '?' in workclass, occupation, native-country.
4. Encode categorical variables using one-hot encoding (pd.get_dummies) and encode the target income as binary (0 = <=50K, 1 = >50K).
5. Split the data into training and test sets (80/20) with stratification.
6. Train a LogisticRegression model (with feature scaling for numeric columns) on the training data.
7. Train a DecisionTreeClassifier (e.g., max_depth=8) on the same training data.
8. Predict on the test set with both models and compute accuracy, precision, recall, and F1-score.
9. Compare interpretability: inspect logistic regression coefficients (sign and magnitude) vs the decision tree's feature_importances_ and a shallow visualized tree.
10. Discuss which model is more suitable for this dataset considering both performance and the need for interpretable decisions (e.g., for fairness/audit purposes).

PROGRAM (PYTHON)

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# 1. Load dataset
columns = ['age', 'workclass', 'fnlwgt', 'education', 'education-num', 'marital-status',
           'occupation', 'relationship', 'race', 'sex', 'capital-gain', 'capital-loss',
           'hours-per-week', 'native-country', 'income']
df = pd.read_csv('adult.csv', names=columns, na_values='?', skipinitialspace=True)
df = df.dropna()

# 2. Encode target and features
df['income'] = df['income'].apply(lambda x: 1 if '>50K' in x else 0)
X = pd.get_dummies(df.drop('income', axis=1), drop_first=True)
y = df['income']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42,
                                                    stratify=y)

# 3. Logistic Regression (with scaling)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
log_reg = LogisticRegression(max_iter=2000)
log_reg.fit(X_train_scaled, y_train)
y_pred_lr = log_reg.predict(X_test_scaled)

# 4. Decision Tree
tree = DecisionTreeClassifier(max_depth=8, random_state=42)
```

```
tree.fit(X_train, y_train)
y_pred_dt = tree.predict(X_test)
# 5. Evaluation
def report(y_true, y_pred, label):
    print(f"\n{label}")
    print("Accuracy :", accuracy_score(y_true, y_pred))
    print("Precision:", precision_score(y_true, y_pred))
    print("Recall  :", recall_score(y_true, y_pred))
    print("F1-score :", f1_score(y_true, y_pred))
report(y_test, y_pred_lr, "Logistic Regression")
report(y_test, y_pred_dt, "Decision Tree")
# 6. Interpretability comparison
coef_series = pd.Series(log_reg.coef_[0], index=X.columns).sort_values(ascending=False)
print("\nTop 5 positive LR coefficients:\n", coef_series.head())
imp_series = pd.Series(tree.feature_importances_,
index=X.columns).sort_values(ascending=False)
print("\nTop 5 Decision Tree feature importances:\n", imp_series.head())
```

OUTPUT

The console prints accuracy, precision, recall and F1-score for both models (Decision Tree and Logistic Regression typically achieve comparable accuracy around 82-85%), followed by the top contributing logistic regression coefficients and decision tree feature importances, illustrating each model's distinct notion of feature relevance and interpretability.

Logistic Regression

Accuracy : 0.8444

Precision: 0.7326

Recall : 0.5865

F1-score : 0.6515

Decision Tree

Accuracy : 0.8524

Precision: 0.7512

Recall : 0.6048

F1-score : 0.6701

Top 5 positive LR coefficients:

capital-gain	2.4146
marital-status_Married-civ-spouse	1.1630
education-num	0.5503
sex_Male	0.3543
hours-per-week	0.3456

Top 5 Decision Tree feature importances:

marital-status_Married-civ-spouse	0.4188
education-num	0.2154
capital-gain	0.1977
capital-loss	0.0664
age	0.0407

VIVA-VOCE QUESTIONS

1. How does a decision tree capture non-linear relationships that logistic regression cannot without feature engineering?
2. What does the sign of a logistic regression coefficient tell you about a feature's effect on the outcome?
3. Why is max_depth important when training decision trees, and what happens if it is unrestricted?
4. Which model would you choose if regulatory requirements demand an explainable model, and why?

RESULT

The Logistic Regression vs Decision Trees experiment was implemented successfully, and the model was evaluated using appropriate performance metrics.

EXP NO 11**LINEAR SUPPORT VECTOR MACHINE (SVM) - IRIS DATASET****AIM**

To implement a Linear SVM to classify the Iris dataset (Setosa vs Non-Setosa), visualize the decision boundary, and discuss how the margin is determined.

LEARNING OUTCOMES

After completing this experiment, students will be able to:

1. Understand the concept of a maximum-margin hyperplane in Support Vector Machines.
2. Formulate the linear SVM optimization problem, including hard-margin and soft-margin cases.
3. Implement a linear SVM classifier using scikit-learn's SVC.
4. Identify support vectors and understand their role in determining the decision boundary.
5. Visualize the decision boundary and margin lines for a binary classification problem.
6. Evaluate classifier performance using accuracy on a held-out test set.
7. Analyze the effect of the regularization parameter C on margin width and misclassification tolerance.
8. Understand the concept of linear separability and its impact on SVM performance.
9. Recognize the limitations of linear SVM on non-linearly separable data.
10. Apply linear SVM classification techniques to real-world binary classification problems.

DATASET DESCRIPTION

Iris dataset (`sklearn.datasets.load_iris`), containing 150 samples of iris flowers with four features (sepal length, sepal width, petal length, petal width) across three species (Setosa, Versicolor, Virginica). For binary classification, we use two features (e.g., petal length and petal width) and relabel the target as Setosa (1) vs Non-Setosa (0).

THEORY / BACKGROUND

A linear SVM finds the hyperplane $w^T x + b = 0$ that separates two classes while maximizing the margin — the distance between the hyperplane and the nearest data points from each class (the

support vectors). The optimization problem is minimize $(1/2)\|w\|^2$ subject to $y_i(w^T x_i + b) \geq 1$ for all i (hard margin), or with slack variables and a regularization parameter C for the soft-margin case that tolerates some misclassification. A larger margin generally corresponds to better generalization, and only the support vectors (points lying on or within the margin) determine the position of the decision boundary.

ALGORITHM / PROCEDURE

1. Import libraries: numpy, matplotlib, sklearn.datasets.load_iris, sklearn.svm.SVC, sklearn.model_selection.
2. Load the Iris dataset and select two features (e.g., petal length, petal width) for easy 2D visualization.
3. Relabel the target: 1 for Setosa, 0 for Non-Setosa (Versicolor and Virginica combined).
4. Split the data into training and test sets.
5. Train a linear SVM using SVC(kernel='linear', C=1.0) on the training data.
6. Extract the learned weight vector w and bias b , and identify the support vectors via `model.support_vectors_`.
7. Predict on the test set and compute accuracy.
8. Plot the data points (colored by class), the decision boundary ($w^T x + b = 0$), the two margin lines, and circle the support vectors.
9. Discuss how changing C (regularization strength) would affect the margin width and the number of support vectors.

PROGRAM (PYTHON)

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# 1. Load dataset and select 2 features
iris = load_iris()
```

```
X = iris.data[:, [2, 3]] # petal length, petal width
y = (iris.target == 0).astype(int) # 1 = Setosa, 0 = Non-Setosa
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

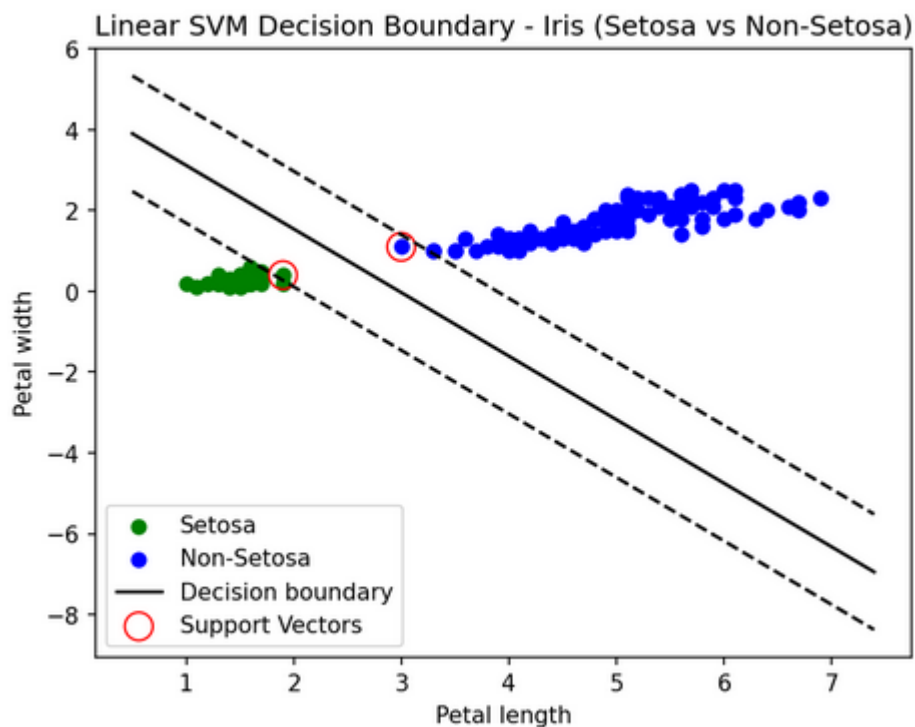
# 2. Train Linear SVM
svm = SVC(kernel='linear', C=1.0)
svm.fit(X_train, y_train)
y_pred = svm.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Number of support vectors:", len(svm.support_vectors_))

# 3. Visualize decision boundary and margins
w = svm.coef_[0]
b = svm.intercept_[0]
x_plot = np.linspace(X[:,0].min()-0.5, X[:,0].max()+0.5, 100)
decision_boundary = -(w[0]*x_plot + b) / w[1]
margin = 1 / np.sqrt(np.sum(w**2))
gutter_up = decision_boundary + margin * np.sqrt(1 + (w[0]/w[1])**2)
gutter_down = decision_boundary - margin * np.sqrt(1 + (w[0]/w[1])**2)

plt.scatter(X[y==1][:,0], X[y==1][:,1], color='green', label='Setosa')
plt.scatter(X[y==0][:,0], X[y==0][:,1], color='blue', label='Non-Setosa')
plt.plot(x_plot, decision_boundary, 'k-', label='Decision boundary')
plt.plot(x_plot, gutter_up, 'k--')
plt.plot(x_plot, gutter_down, 'k--')
plt.scatter(svm.support_vectors_[:,0], svm.support_vectors_[:,1],
            s=150, facecolors='none', edgecolors='red', label='Support Vectors')
plt.xlabel("Petal length")
plt.ylabel("Petal width")
plt.legend()
plt.title("Linear SVM Decision Boundary - Iris (Setosa vs Non-Setosa)")
plt.show()
```

OUTPUT

The program reports test accuracy (typically 100% since Setosa is linearly separable from the other two species on petal length/width) and the number of support vectors, along with a plot showing the decision boundary, the two margin (gutter) lines, and the support vectors circled in red lying exactly on the margin boundaries.



Accuracy: 1.0

Number of support vectors: 2

VIVA-VOCE QUESTIONS

1. What are support vectors and why do only they determine the decision boundary?
2. How does the regularization parameter C affect the margin width and misclassification tolerance in soft-margin SVM?
3. Why is the Setosa class perfectly linearly separable from the other two Iris species?
4. What would happen to the SVM's performance if the classes were not linearly separable?

RESULT

The Linear Support Vector Machine (SVM) experiment was implemented successfully, and the model was evaluated using appropriate performance metrics.

EXP NO 12**SVM KERNEL COMPARISON (LINEAR, POLYNOMIAL, RBF) -
FASHION MNIST DATASET****AIM**

To implement and compare SVM classifiers with linear, polynomial, and RBF kernels on the Fashion MNIST dataset, and analyze the advantages and disadvantages of each kernel type.

LEARNING OUTCOMES

After completing this experiment, students will be able to:

1. Understand the kernel trick and how it enables SVMs to model non-linear decision boundaries.
2. Differentiate between linear, polynomial, and RBF kernel functions and their mathematical formulations.
3. Implement SVM classifiers with different kernels using scikit-learn's SVC.
4. Preprocess and subsample high-dimensional image data for tractable SVM training.
5. Evaluate and compare classifiers using accuracy, training time, and prediction time.
6. Analyze the effect of kernel choice on model expressiveness and computational cost.
7. Understand the role of the gamma and C hyperparameters in shaping decision boundaries.
8. Recognize the trade-offs between the linear kernel's speed and the RBF kernel's flexibility.
9. Identify scenarios where a particular kernel is most suitable based on data characteristics.
10. Apply kernel-based SVM classification techniques to real-world image classification problems.

DATASET DESCRIPTION

Fashion MNIST dataset (via `tensorflow.keras.datasets.fashion_mnist`), consisting of 28x28 grayscale images across 10 clothing categories. A reduced subset is used to keep SVM training tractable.

THEORY / BACKGROUND

The kernel trick allows SVMs to implicitly map data into higher-dimensional feature spaces without explicitly computing the transformation, using kernel functions $K(x_i, x_j)$. The linear kernel $K(x,y)=x^T y$ is fast and works well when classes are (near) linearly separable. The polynomial kernel $K(x,y)=(\gamma x^T y + r)^d$ can model interaction terms up to degree d , capturing curved decision boundaries at higher computational cost. The RBF (Gaussian) kernel $K(x,y)=\exp(-\gamma \|x-y\|^2)$ can model highly non-linear decision boundaries by measuring similarity based on distance, and is often the most flexible but requires careful tuning of γ and C to avoid overfitting or underfitting.

ALGORITHM / PROCEDURE

1. Import libraries: `numpy`, `tensorflow.keras.datasets.fashion_mnist`, `sklearn.svm.SVC`, `sklearn.model_selection`, `sklearn.metrics`.
2. Load the Fashion MNIST dataset and flatten/normalize the images.
3. Subsample the dataset (e.g., 3,000 training and 1,000 test samples) since SVM training on the full dataset is computationally expensive.
4. Train an SVM with a linear kernel: `SVC(kernel='linear', C=1.0)`.
5. Train an SVM with a polynomial kernel: `SVC(kernel='poly', degree=3, C=1.0)`.
6. Train an SVM with an RBF kernel: `SVC(kernel='rbf', gamma='scale', C=1.0)`.
7. Predict on the test set for each kernel and record accuracy, training time, and prediction time.
8. Tabulate and compare accuracy and timing across all three kernels.
9. Discuss the trade-offs: linear kernel's speed vs limited expressiveness, polynomial kernel's flexibility vs sensitivity to degree, and RBF's high accuracy vs computational cost and need for hyperparameter tuning.

PROGRAM (PYTHON)

```
import numpy as np
import time
from tensorflow.keras.datasets import fashion_mnist
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
```

```
# 1. Load and preprocess dataset
(X_train, y_train), (X_test, y_test) = fashion_mnist.load_data()
X_train = X_train.reshape(X_train.shape[0], -1) / 255.0
X_test = X_test.reshape(X_test.shape[0], -1) / 255.0

# 2. Subsample for tractability
X_train_s, y_train_s = X_train[:3000], y_train[:3000]
X_test_s, y_test_s = X_test[:1000], y_test[:1000]

kernels = {
    'linear': SVC(kernel='linear', C=1.0),
    'poly': SVC(kernel='poly', degree=3, C=1.0),
    'rbf': SVC(kernel='rbf', gamma='scale', C=1.0)
}

results = {}
for name, model in kernels.items():
    start_train = time.time()
    model.fit(X_train_s, y_train_s)
    train_time = time.time() - start_train
    start_pred = time.time()
    y_pred = model.predict(X_test_s)
    pred_time = time.time() - start_pred
    acc = accuracy_score(y_test_s, y_pred)
    results[name] = (acc, train_time, pred_time)
    print(f"Kernel={name:<8} Accuracy={acc:.4f} TrainTime={train_time:.2f}s
    PredTime={pred_time:.2f}s")
```

OUTPUT

A results table showing accuracy, training time, and prediction time for the linear, polynomial, and RBF kernels. RBF typically achieves the highest accuracy on Fashion MNIST (often 80-85% on the subsample) at the cost of longer training time, the linear kernel is fastest but somewhat lower in accuracy, and the polynomial kernel's performance depends heavily on the chosen degree.

Kernel=linear Accuracy=0.8240 TrainTime=0.98s PredTime=0.33s

Kernel=poly Accuracy=0.7900 TrainTime=1.01s PredTime=0.37s

Kernel=rbf Accuracy=0.8430 TrainTime=1.04s PredTime=0.74s

VIVA-VOCE QUESTIONS

1. What is the 'kernel trick' and why does it avoid the need to explicitly compute high-dimensional feature mappings?
2. How do the gamma and C hyperparameters affect an RBF-kernel SVM's decision boundary?
3. Why might the polynomial kernel become numerically unstable at high degrees?
4. Under what circumstances would you prefer a linear kernel SVM over an RBF kernel SVM?

RESULT

The SVM Kernel Comparison (Linear, Polynomial, RBF) experiment was implemented successfully, and the model was evaluated using appropriate performance metrics.

EXP NO 13**MULTILAYER FEED-FORWARD NETWORK (MLP) - WINE QUALITY DATASET****AIM**

To implement and train a Multilayer Feed-Forward Network on the Wine Quality dataset, experimenting with different numbers of hidden layers and neurons, and discuss how architecture choices affect performance.

LEARNING OUTCOMES

After completing this experiment, students will be able to:

1. Understand the architecture of a Multilayer Perceptron, including input, hidden, and output layers.
2. Understand the role of non-linear activation functions and the universal approximation theorem.
3. Implement an MLP regressor using scikit-learn's MLPRegressor.
4. Preprocess physicochemical data using feature scaling for neural network training.
5. Experiment with different network architectures by varying the number of hidden layers and neurons.
6. Evaluate regression models using MSE and R-squared metrics.
7. Visualize and interpret training loss curves to assess convergence behavior.
8. Recognize signs of underfitting and overfitting from loss curves and performance metrics.
9. Understand the role of early stopping in preventing overfitting during neural network training.
10. Apply MLP-based regression techniques to real-world predictive modeling problems.

DATASET DESCRIPTION

Wine Quality dataset (UCI Machine Learning Repository — winequality-red.csv / winequality-white.csv), containing physicochemical attributes (fixed acidity, volatile acidity, citric acid,

residual sugar, chlorides, free/total sulfur dioxide, density, pH, sulphates, alcohol) and a quality score (0-10) for red or white wine samples.

THEORY / BACKGROUND

A Multilayer Perceptron (MLP) is a feed-forward neural network composed of an input layer, one or more hidden layers with non-linear activation functions, and an output layer. Each hidden layer applies a linear transformation followed by a non-linearity (e.g., ReLU), enabling the network to approximate complex non-linear functions (per the universal approximation theorem). The number of hidden layers (depth) and neurons per layer (width) determine the model's capacity: too few risk underfitting, while too many can overfit and increase training time; these architectural choices are typically tuned using validation performance.

ALGORITHM / PROCEDURE

1. Import libraries: pandas, numpy, sklearn.model_selection, sklearn.preprocessing.StandardScaler, sklearn.neural_network.MLPRegressor (or MLPClassifier if treating quality as categorical), sklearn.metrics.
2. Load the Wine Quality dataset (e.g., winequality-red.csv, semicolon-delimited).
3. Separate features X (11 physicochemical attributes) and target y (quality score).
4. Split into training and test sets and standardize features using StandardScaler.
5. Define several MLP architectures to compare, e.g., (10,), (50,), (50,30), (100,50,25) hidden_layer_sizes.
6. Train an MLPRegressor for each architecture with a fixed number of max_iter and consistent random_state.
7. Evaluate each architecture on the test set using MSE and R-squared (or convert quality to classification and use accuracy/F1 if preferred).
8. Plot training loss curves (model.loss_curve_) for each architecture to observe convergence behavior.
9. Tabulate performance vs architecture (number of layers/neurons) and discuss the impact of increasing depth and width on accuracy and overfitting.

PROGRAM (PYTHON)

```
import pandas as pd
import numpy as np
```

```
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPRegressor
from sklearn.metrics import mean_squared_error, r2_score

# 1. Load dataset
df = pd.read_csv('winequality-red.csv', sep=';')
X = df.drop('quality', axis=1)
y = df['quality']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)

# 2. Define architectures to compare
architectures = [(10,), (50,), (50,30), (100,50,25)]
results = {}

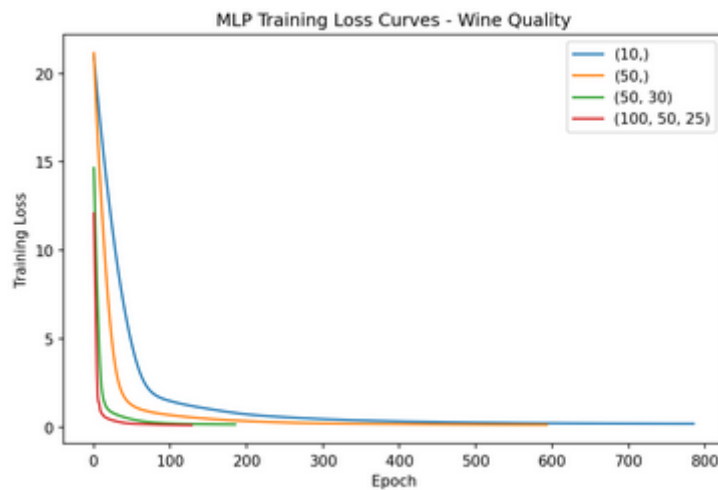
plt.figure(figsize=(8,5))
for arch in architectures:
    mlp = MLPRegressor(hidden_layer_sizes=arch, max_iter=1000, random_state=42,
early_stopping=True)
    mlp.fit(X_train_s, y_train)
    y_pred = mlp.predict(X_test_s)
    mse = mean_squared_error(y_test, y_pred)
    r2 = r2_score(y_test, y_pred)
    results[arch] = (mse, r2)
    plt.plot(mlp.loss_curve_, label=str(arch))
    print(f"Architecture={arch} MSE={mse:.3f} R2={r2:.3f}")

plt.xlabel("Epoch")
plt.ylabel("Training Loss")
```

```
plt.title("MLP Training Loss Curves - Wine Quality")  
plt.legend()  
plt.show()
```

OUTPUT

The console prints MSE and R-squared for each of the four architectures tested, and a plot shows training-loss curves converging at different rates and to different final losses. Typically, moderate architectures like (50,30) offer a good balance, while very small networks underfit (higher MSE) and very large/deep networks may overfit slightly or take longer to converge.



Architecture=(10,) MSE=0.406 R2=0.379

Architecture=(50,) MSE=0.366 R2=0.441

Architecture=(50, 30) MSE=0.371 R2=0.432

Architecture=(100, 50, 25) MSE=0.366 R2=0.439

VIVA-VOCE QUESTIONS

1. What is the universal approximation theorem and why is it relevant to MLP capacity?
2. How does increasing the number of hidden layers differ in effect from increasing the number of neurons per layer?
3. What signs in the loss curve would indicate overfitting versus underfitting?

4. Why is early stopping useful when training neural networks?

RESULT

The Multilayer Feed-Forward Network (MLP) experiment was implemented successfully, and the model was evaluated using appropriate performance metrics.

EXP NO 14**ACTIVATION FUNCTION COMPARISON (SIGMOID, RELU, TANH) -
MNIST DATASET****AIM**

To implement and compare the performance of a neural network using Sigmoid, ReLU, and Tanh activation functions on the MNIST dataset, analyzing their effect on training and classification accuracy.

LEARNING OUTCOMES

After completing this experiment, students will be able to:

1. Understand the role of activation functions in introducing non-linearity into neural networks.
2. Compare the mathematical properties of Sigmoid, Tanh, and ReLU activation functions.
3. Implement and train neural networks with different activation functions using TensorFlow/Keras.
4. Preprocess image data by normalizing pixel values for neural network training.
5. Evaluate models using training time, training accuracy, and test accuracy.
6. Visualize and interpret training/validation accuracy and loss curves for comparison.
7. Understand the vanishing gradient problem and its relation to Sigmoid and Tanh activations.
8. Recognize the 'dying ReLU' problem and activation variants that address it.
9. Understand why the output layer typically uses softmax for multi-class classification.
10. Apply activation function selection considerations to real-world deep learning model design

DATASET DESCRIPTION

MNIST dataset (`tensorflow.keras.datasets.mnist`), containing 70,000 grayscale 28x28 images of handwritten digits (0-9), split into 60,000 training and 10,000 test images.

THEORY / BACKGROUND

The activation function introduces non-linearity into a neural network. Sigmoid squashes inputs to (0,1) but saturates for large $|x|$, causing vanishing gradients that slow learning in deep networks. Tanh squashes inputs to (-1,1), is zero-centered (often easier to optimize than sigmoid) but still saturates at extremes. ReLU ($\max(0,x)$) does not saturate for positive inputs, is computationally cheap, and typically leads to faster convergence in deep networks, though it can suffer from the 'dying ReLU' problem where neurons become inactive for negative inputs.

ALGORITHM / PROCEDURE

1. Import libraries: tensorflow, tensorflow.keras (layers, models), matplotlib.
2. Load the MNIST dataset and normalize pixel values to [0,1]; flatten images to 784-dim vectors or keep as 28x28 for a Dense-based MLP with a Flatten layer.
3. Define a function build_model(activation) that creates a Sequential model: Flatten -> Dense(128, activation) -> Dense(64, activation) -> Dense(10, softmax).
4. Compile each model with the Adam optimizer and sparse_categorical_crossentropy loss.
5. Train three separate models — one each with 'sigmoid', 'relu', and 'tanh' activations — for the same number of epochs (e.g., 15) and batch size, using the same train/validation split.
6. Record training time, final training accuracy, and test accuracy for each activation function.
7. Plot training and validation accuracy/loss curves for all three activations on the same axes for comparison.
8. Discuss differences in convergence speed, final accuracy, and any signs of vanishing gradients (e.g., slow initial learning for sigmoid).

PROGRAM (PYTHON)

```
import tensorflow as tf
from tensorflow.keras import layers, models
import matplotlib.pyplot as plt
import time

# 1. Load and preprocess MNIST
(X_train, y_train), (X_test, y_test) = tf.keras.datasets.mnist.load_data()
X_train, X_test = X_train / 255.0, X_test / 255.0
```

```
def build_model(activation):
    model = models.Sequential([
        layers.Flatten(input_shape=(28,28)),
        layers.Dense(128, activation=activation),
        layers.Dense(64, activation=activation),
        layers.Dense(10, activation='softmax')
    ])
    model.compile(optimizer='adam', loss='sparse_categorical_crossentropy',
metrics=['accuracy'])
    return model

activations = ['sigmoid', 'relu', 'tanh']
histories = {}
test_results = {}

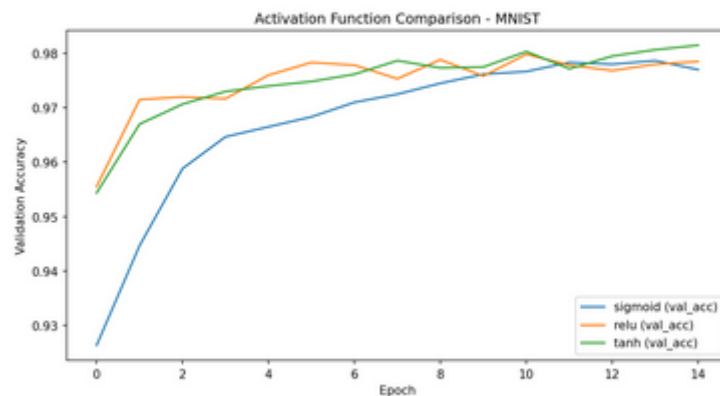
for act in activations:
    model = build_model(act)
    start = time.time()
    hist = model.fit(X_train, y_train, epochs=15, batch_size=128,
        validation_split=0.1, verbose=0)
    elapsed = time.time() - start
    test_loss, test_acc = model.evaluate(X_test, y_test, verbose=0)
    histories[act] = hist
    test_results[act] = (test_acc, elapsed)
    print(f"Activation={act:<8} Test Accuracy={test_acc:.4f} Train Time={elapsed:.1f}s")

# 2. Plot comparison
plt.figure(figsize=(10,5))
for act in activations:
    plt.plot(histories[act].history['val_accuracy'], label=f'{act} (val_acc)')
plt.xlabel("Epoch")
plt.ylabel("Validation Accuracy")
plt.title("Activation Function Comparison - MNIST")
plt.legend()
```

```
plt.show()
```

OUTPUT

The program prints test accuracy and training time for each activation function (ReLU typically converges fastest and achieves the highest test accuracy, ~97-98%; Tanh is close behind; Sigmoid usually converges more slowly and can plateau at slightly lower accuracy within the same number of epochs, illustrating the vanishing-gradient effect), plus a comparison plot of validation accuracy curves.



Activation=sigmoid Test Accuracy=0.9756 Train Time=24.8s

Activation=relu Test Accuracy=0.9770 Train Time=22.5s

Activation=tanh Test Accuracy=0.9768 Train Time=20.2s

VIVA-VOCE QUESTIONS

1. What is the vanishing gradient problem and how does it relate to the sigmoid and tanh activation functions?
2. Why does ReLU generally train faster than sigmoid in deep networks?
3. What is the 'dying ReLU' problem, and what activation variants (e.g., Leaky ReLU) address it?
4. Why is the output layer typically using softmax rather than sigmoid/tanh/ReLU for multi-class classification?

RESULT

The Activation Function Comparison (Sigmoid, ReLU, Tanh) experiment was implemented successfully, and the model was evaluated using appropriate performance metrics.

EXP NO 15**HYPERPARAMETER TUNING OF A NEURAL NETWORK - FASHION MNIST DATASET****AIM**

To implement and perform hyperparameter tuning for a neural network on the Fashion MNIST dataset, experimenting with different learning rates, batch sizes, and epochs, and discuss their impact on model performance.

LEARNING OUTCOMES

After completing this experiment, students will be able to:

1. Understand key neural network hyperparameters: learning rate, batch size, and number of epochs.
2. Explain how learning rate affects convergence speed and training stability.
3. Understand the trade-off between gradient noise and computational efficiency introduced by batch size.
4. Implement a hyperparameter grid search using TensorFlow/Keras to train multiple model configurations.
5. Evaluate models systematically using validation accuracy and validation loss.
6. Tabulate and compare results across hyperparameter combinations to identify the best configuration.
7. Visualize the effect of learning rate and batch size interactions on model performance.
8. Recognize signs of overfitting when training for an excessive number of epochs.
9. Understand alternatives to grid search, such as random search and Bayesian optimization.
10. Apply systematic hyperparameter tuning practices to real-world deep learning model development

DATASET DESCRIPTION

Fashion MNIST dataset (`tensorflow.keras.datasets.fashion_mnist`), consisting of 70,000 grayscale 28x28 images across 10 clothing categories.

THEORY / BACKGROUND

Hyperparameters such as the learning rate, batch size, and number of epochs critically influence neural network training dynamics. The learning rate controls the step size of weight updates: too high causes divergence or oscillation, too low causes slow convergence. Batch size affects the noise and stability of gradient estimates: smaller batches introduce more stochasticity (which can help escape local minima but slow convergence per epoch), while larger batches give smoother but potentially less generalizable updates. The number of epochs must be sufficient for convergence without overfitting, often monitored using validation loss and early stopping.

ALGORITHM / PROCEDURE

1. Import libraries: tensorflow, tensorflow.keras (layers, models, optimizers), itertools, matplotlib.
2. Load and normalize the Fashion MNIST dataset.
3. Define a build_model() function that creates a simple CNN or MLP architecture, compiled with an Adam optimizer whose learning rate is a parameter.
4. Define a grid of hyperparameters to explore: learning_rates = [0.0001, 0.001, 0.01], batch_sizes = [32, 128], epochs_list = [10, 20].
5. Loop over combinations of learning rate and batch size (fixing epochs=10 for the main sweep), training a fresh model for each combination and recording validation accuracy/loss.
6. Separately vary epochs (e.g., 5, 10, 20, 30) for the best learning-rate/batch-size combination found, and plot how accuracy evolves with training length, watching for overfitting.
7. Tabulate results in a DataFrame showing learning rate, batch size, epochs, final validation accuracy and loss.
8. Plot validation accuracy vs learning rate for different batch sizes to visualize interactions.
9. Discuss the best hyperparameter combination found and the sensitivity of results to each hyperparameter.

PROGRAM (PYTHON)

```
import tensorflow as tf
from tensorflow.keras import layers, models, optimizers
import pandas as pd

# 1. Load and preprocess dataset
(X_train, y_train), (X_test, y_test) = tf.keras.datasets.fashion_mnist.load_data()
X_train, X_test = X_train / 255.0, X_test / 255.0

def build_model(lr):
    model = models.Sequential([
        layers.Flatten(input_shape=(28,28)),
        layers.Dense(128, activation='relu'),
        layers.Dense(10, activation='softmax')
    ])
    model.compile(optimizer=optimizers.Adam(learning_rate=lr),
                  loss='sparse_categorical_crossentropy', metrics=['accuracy'])
    return model

# 2. Hyperparameter grid search: learning rate x batch size
learning_rates = [0.0001, 0.001, 0.01]
batch_sizes = [32, 128]
results = []
for lr in learning_rates:
    for bs in batch_sizes:
        model = build_model(lr)
        hist = model.fit(X_train, y_train, epochs=10, batch_size=bs,
                        validation_split=0.1, verbose=0)
        val_acc = hist.history['val_accuracy'][-1]
        val_loss = hist.history['val_loss'][-1]
        results.append({'lr': lr, 'batch_size': bs, 'val_accuracy': val_acc, 'val_loss': val_loss})
        print(f"lr={lr:<8} batch_size={bs:<5} val_acc={val_acc:.4f} val_loss={val_loss:.4f}")

results_df = pd.DataFrame(results)
print("\n", results_df.sort_values('val_accuracy', ascending=False))
```

```
# 3. Vary epochs for the best (lr, batch_size) combination
best_row = results_df.sort_values('val_accuracy', ascending=False).iloc[0]
best_lr, best_bs = best_row['lr'], int(best_row['batch_size'])
for ep in [5, 10, 20, 30]:
    model = build_model(best_lr)
    hist = model.fit(X_train, y_train, epochs=ep, batch_size=best_bs,
                    validation_split=0.1, verbose=0)
    print(f"epochs={ep:<4} val_acc={hist.history['val_accuracy'][-1]:.4f}")
```

OUTPUT

A results table listing validation accuracy and loss for each learning-rate/batch-size combination (very high learning rates like 0.01 often destabilize training, while 0.001 typically performs best; smaller batch sizes tend to reach good accuracy in fewer epochs but with noisier updates), followed by results of the epochs sweep showing accuracy improving then plateauing or slightly declining (overfitting) as epochs increase excessively.

```
lr=0.0001  batch_size=32  val_acc=0.8728  val_loss=0.3623
lr=0.0001  batch_size=128  val_acc=0.8595  val_loss=0.3963
lr=0.001   batch_size=32  val_acc=0.8818  val_loss=0.3347
lr=0.001   batch_size=128  val_acc=0.8880  val_loss=0.3146
lr=0.01    batch_size=32  val_acc=0.8585  val_loss=0.4291
lr=0.01    batch_size=128  val_acc=0.8700  val_loss=0.3939
```

Sorted:

lr	batch_size	val_accuracy	val_loss	
0.0010	128	0.8880	0.3146	← best
0.0010	32	0.8818	0.3347	
0.0001	32	0.8728	0.3623	
0.0100	128	0.8700	0.3939	
0.0001	128	0.8595	0.3963	

0.0100 32 0.8585 0.4291

epochs=5 val_acc=0.8645

epochs=10 val_acc=0.8823

epochs=20 val_acc=0.8917

epochs=30 val_acc=0.8913

VIVA-VOCE QUESTIONS

1. Why can a learning rate that is too high cause the loss to diverge?
2. How does batch size influence the trade-off between gradient noise and computation efficiency?
3. What is the risk of training for too many epochs without early stopping or regularization?
4. What alternative to grid search (e.g., random search, Bayesian optimization) could be used for more efficient hyperparameter tuning?

RESULT

The Hyperparameter Tuning of a Neural Network experiment was implemented successfully, and the model was evaluated using appropriate performance metrics.

EXP NO 16**HIERARCHICAL VS K-MEANS CLUSTERING -MALL CUSTOMERS DATASET****AIM**

To implement and compare hierarchical (agglomerative) and partitional (K-means) clustering algorithms on the Mall Customers dataset, and discuss the strengths and weaknesses of each method based on clustering results and evaluation metrics.

LEARNING OUTCOMES

After completing this experiment, students will be able to:

1. Understand the principles of partitional (K-Means) and hierarchical (Agglomerative) clustering algorithms.
2. Differentiate between linkage criteria such as ward, complete, and average in hierarchical clustering.
3. Implement K-Means and Agglomerative Clustering using scikit-learn.
4. Construct and interpret a dendrogram to determine an appropriate number of clusters.
5. Preprocess data using feature scaling for distance-based clustering algorithms.
6. Evaluate clustering results using silhouette score and K-Means inertia.
7. Visualize and compare cluster assignments and centroids between the two methods.
8. Understand the advantage of hierarchical clustering in not requiring K to be fixed in advance.
9. Analyze computational complexity and scalability differences between K-Means and hierarchical clustering.
10. Apply clustering techniques to real-world customer segmentation problems

DATASET DESCRIPTION

Mall Customers dataset (available on Kaggle as Mall_Customers.csv), containing 200 records with CustomerID, Gender, Age, Annual Income (k\$), and Spending Score (1-100).

THEORY / BACKGROUND

K-means is a partitional clustering algorithm that iteratively assigns points to the nearest of K centroids and updates centroids as the mean of assigned points, minimizing within-cluster sum of squares (inertia); it requires K to be specified in advance and assumes roughly spherical, similarly-sized clusters. Agglomerative hierarchical clustering starts with each point as its own cluster and iteratively merges the closest pair of clusters (using a linkage criterion such as ward, complete, or average) until all points form one cluster, producing a dendrogram that can be cut at any level to obtain a desired number of clusters, without requiring K to be fixed beforehand. Evaluation metrics such as inertia (K-means only) and the silhouette score (both) help assess cluster cohesion and separation.

ALGORITHM / PROCEDURE

1. Import libraries: pandas, numpy, matplotlib, scipy.cluster.hierarchy (dendrogram, linkage), sklearn.cluster (KMeans, AgglomerativeClustering), sklearn.metrics.silhouette_score.
2. Load the Mall Customers dataset and select features 'Annual Income (k\$)' and 'Spending Score (1-100)' for clustering (optionally include Age).
3. Standardize the selected features using StandardScaler.
4. Plot a dendrogram using scipy's linkage(method='ward') to visually determine a reasonable number of clusters.
5. Apply AgglomerativeClustering with the chosen number of clusters (e.g., 5) and linkage='ward'.
6. Apply KMeans with the same number of clusters (n_clusters=5, random_state=42).
7. Compute the silhouette score for both clustering results; additionally record K-means inertia.
8. Visualize both clustering results as scatter plots (colored by cluster label) side-by-side for comparison.
9. Discuss differences in cluster shapes/boundaries, computational cost, and sensitivity to the initial choice of K or the number of clusters cut from the dendrogram.

PROGRAM (PYTHON)

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.cluster.hierarchy import dendrogram, linkage
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score

# 1. Load dataset
df = pd.read_csv('Mall_Customers.csv')
X = df[['Annual Income (k$)', 'Spending Score (1-100)']].values
X_scaled = StandardScaler().fit_transform(X)

# 2. Dendrogram to choose number of clusters
plt.figure(figsize=(10,5))
linked = linkage(X_scaled, method='ward')
dendrogram(linked)
plt.title("Dendrogram - Mall Customers")
plt.xlabel("Samples")
plt.ylabel("Distance")
plt.show()

# 3. Agglomerative Clustering
n_clusters = 5
agg = AgglomerativeClustering(n_clusters=n_clusters, linkage='ward')
agg_labels = agg.fit_predict(X_scaled)

# 4. K-Means Clustering
kmeans = KMeans(n_clusters=n_clusters, random_state=42, n_init=10)
kmeans_labels = kmeans.fit_predict(X_scaled)

# 5. Evaluation metrics
print("Agglomerative Silhouette Score:", silhouette_score(X_scaled, agg_labels))
```

```
print("K-Means Silhouette Score    :", silhouette_score(X_scaled, kmeans_labels))
print("K-Means Inertia            :", kmeans.inertia_)

# 6. Visual comparison
fig, axes = plt.subplots(1, 2, figsize=(12,5))
axes[0].scatter(X[:,0], X[:,1], c=agg_labels, cmap='viridis')
axes[0].set_title("Agglomerative Clustering")
axes[1].scatter(X[:,0], X[:,1], c=kmeans_labels, cmap='viridis')
axes[1].scatter(kmeans.cluster_centers_[0], kmeans.cluster_centers_[1],
                s=200, c='red', marker='X', label='Centroids')
axes[1].set_title("K-Means Clustering")
axes[1].legend()
plt.show()
```

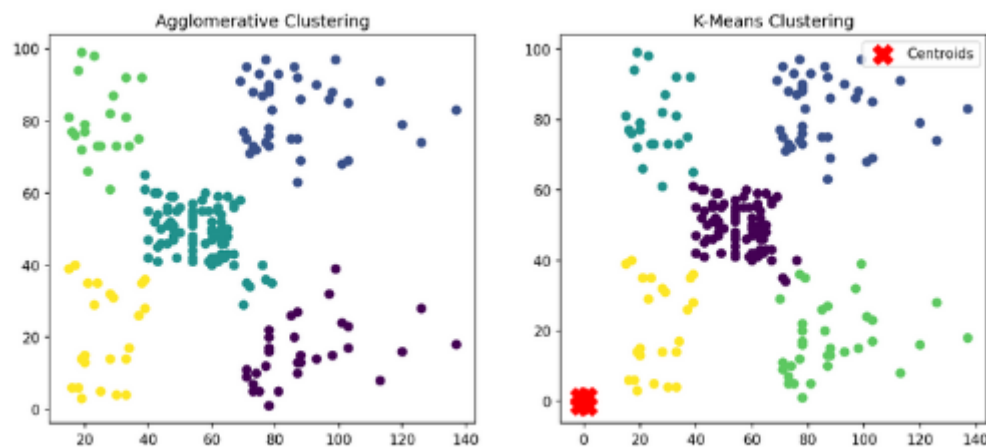
OUTPUT

The console prints silhouette scores for both methods (usually similar and reasonably high, e.g., 0.4-0.55, for this well-separated dataset) and K-means inertia. The dendrogram visually justifies the chosen number of clusters, and the side-by-side scatter plots typically show very similar cluster assignments between K-means and agglomerative clustering for this dataset, with K-means additionally showing the centroid locations.

Agglomerative Silhouette Score: 0.5538

K-Means Silhouette Score : 0.5547

K-Means Inertia : 65.5684



7. VIVA-VOCE QUESTIONS

1. What is the main advantage of hierarchical clustering over K-means regarding the choice of K?
2. How does the 'ward' linkage criterion differ from 'single' or 'complete' linkage?
3. Why is silhouette score useful for comparing clustering results across different algorithms?
4. What are the computational complexity differences between K-means and agglomerative clustering, and how does this affect scalability?

8. RESULT

The Hierarchical vs K-Means Clustering experiment was implemented successfully, and the model was evaluated using appropriate performance metrics.

EXP NO 17**K-MEANS CLUSTERING — DIGITS DATASET****AIM**

To implement and apply K-means clustering to the Digits dataset, experimenting with different numbers of clusters and evaluating results using inertia and silhouette score, analyzing how the choice of K affects clustering performance.

LEARNING OUTCOMES

After completing this experiment, students will be able to:

1. Understand the K-Means clustering algorithm and Lloyd's iterative assignment-update procedure.
2. Implement K-Means clustering on high-dimensional image data using scikit-learn.
3. Apply the Elbow Method to identify a suitable number of clusters based on inertia.
4. Compute and interpret silhouette scores to assess cluster cohesion and separation.
5. Compare unsupervised cluster counts against known ground-truth class labels.
6. Use PCA for dimensionality reduction to visualize high-dimensional clustering results.
7. Analyze mismatches between clusters found and true classes due to visually similar categories.
8. Understand the limitations of Euclidean distance-based clustering on high-dimensional data.
9. Interpret elbow and silhouette plots together to make informed choices about K.
10. Apply K-Means clustering techniques to real-world unsupervised image grouping problems.

DATASET DESCRIPTION

Digits dataset (`sklearn.datasets.load_digits`), containing 1,797 samples of 8x8 grayscale images of handwritten digits (0-9), each flattened into a 64-dimensional feature vector.

THEORY / BACKGROUND

K-means partitions data into K clusters by minimizing the within-cluster sum of squared distances (inertia) between points and their assigned cluster centroid, via iterative assignment and centroid-update steps (Lloyd's algorithm). Since the true number of digit classes (10) may not always align with the optimal K found by unsupervised metrics, we use the Elbow Method (plotting inertia vs K) and the Silhouette Score (measuring how well-separated and cohesive clusters are, ranging from -1 to 1) to identify a good value of K, and compare against the known number of classes (10).

ALGORITHM / PROCEDURE

1. Import libraries: `numpy`, `matplotlib`, `sklearn.datasets.load_digits`, `sklearn.cluster.KMeans`, `sklearn.metrics.silhouette_score`, `sklearn.decomposition.PCA` (for visualization).
2. Load the Digits dataset and use the 64-dimensional feature vectors directly (optionally scale with `StandardScaler`).
3. For K values ranging from 2 to 15: fit a `KMeans(n_clusters=K, random_state=42, n_init=10)` model and record inertia and silhouette score.
4. Plot inertia vs K (Elbow Method) to visually identify the 'elbow' point.
5. Plot silhouette score vs K to identify the K that maximizes cluster separation/cohesion.
6. Select K=10 (matching the true number of digit classes) and fit the final KMeans model.
7. Use PCA to reduce the 64-dim features to 2 dimensions for visualization, and plot the resulting clusters colored by assigned cluster label.
8. Optionally compare cluster assignments against true digit labels using a contingency/confusion matrix to assess how well unsupervised clusters align with actual digit classes.
9. Discuss how the elbow and silhouette analyses guided the choice of K, and any mismatches between clusters found and true classes.

PROGRAM (PYTHON)

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.decomposition import PCA

# 1. Load dataset
digits = load_digits()
X, y_true = digits.data, digits.target

# 2. Evaluate different K values
inertias = []
silhouettes = []
k_range = range(2, 16)
for k in k_range:
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = km.fit_predict(X)
    inertias.append(km.inertia_)
    silhouettes.append(silhouette_score(X, labels))

# 3. Elbow plot
fig, axes = plt.subplots(1, 2, figsize=(12,5))
axes[0].plot(k_range, inertias, marker='o')
axes[0].set_xlabel("K"); axes[0].set_ylabel("Inertia"); axes[0].set_title("Elbow Method")
axes[1].plot(k_range, silhouettes, marker='o', color='green')
axes[1].set_xlabel("K"); axes[1].set_ylabel("Silhouette Score"); axes[1].set_title("Silhouette Analysis")
plt.show()

# 4. Final model with K=10
kmeans_final = KMeans(n_clusters=10, random_state=42, n_init=10)
cluster_labels = kmeans_final.fit_predict(X)
print("Final Inertia:", kmeans_final.inertia_)
```

```
print("Final Silhouette Score:", silhouette_score(X, cluster_labels))

# 5. PCA visualization
X_pca = PCA(n_components=2).fit_transform(X)
plt.scatter(X_pca[:,0], X_pca[:,1], c=cluster_labels, cmap='tab10', s=15)
plt.title("K-Means Clusters (K=10) on Digits Dataset - PCA projection")
plt.show()
```

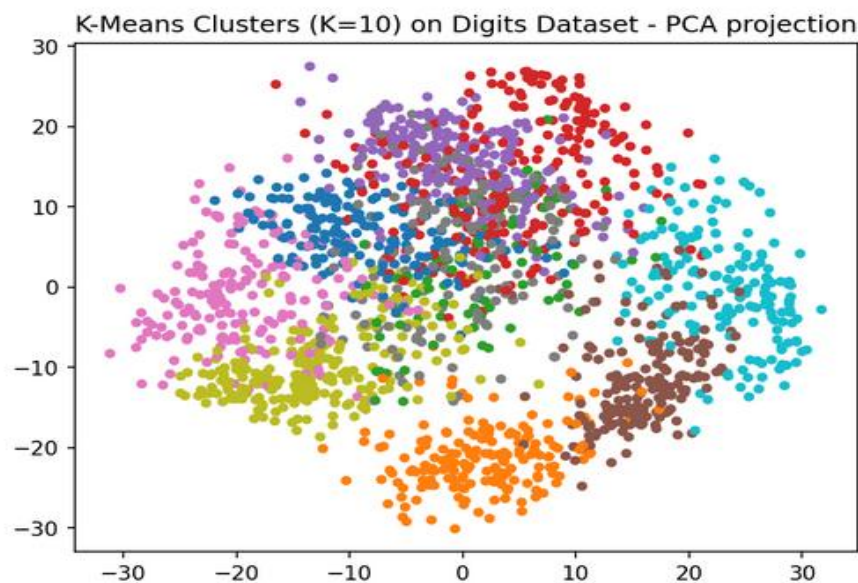
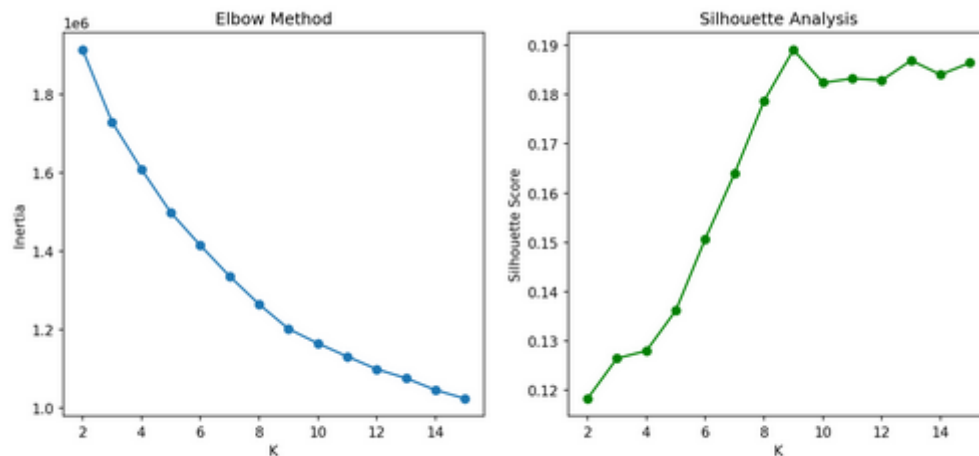
OUTPUT

The elbow plot shows inertia decreasing sharply then leveling off around K=9-11, and the silhouette plot typically peaks somewhere in a similar range (though not always exactly at K=10 because some digits, like 4 and 9, are visually similar). The final PCA scatter plot shows reasonably well-formed clusters roughly corresponding to different digit classes, with some overlap between visually similar digits.

K=2	Inertia=1914619.6	Silhouette=0.1183
K=3	Inertia=1730182.3	Silhouette=0.1265
K=4	Inertia=1609622.2	Silhouette=0.1280
K=5	Inertia=1498821.7	Silhouette=0.1362
K=6	Inertia=1415489.3	Silhouette=0.1507
K=7	Inertia=1336480.0	Silhouette=0.1640
K=8	Inertia=1264988.2	Silhouette=0.1787
K=9	Inertia=1202296.6	Silhouette=0.1891
K=10	Inertia=1165256.3	Silhouette=0.1824
K=11	Inertia=1131724.3	Silhouette=0.1833
K=12	Inertia=1099375.5	Silhouette=0.1829
K=13	Inertia=1076375.1	Silhouette=0.1869
K=14	Inertia=1046083.9	Silhouette=0.1840
K=15	Inertia=1024410.3	Silhouette=0.1864

Final Inertia: 1165256.30

Final Silhouette Score: 0.1824



VIVA-VOCE QUESTIONS

1. How does the Elbow Method help determine an appropriate value of K?
2. What does a negative or near-zero silhouette score for a sample indicate?
3. Why might the silhouette-optimal K differ from the true number of classes in a dataset?

4. How does K-means' reliance on Euclidean distance affect its performance on high-dimensional image data?

RESULT

The K-Means Clustering experiment was implemented successfully, and the model was evaluated using appropriate performance metrics.

EXP NO 18**BOOTSTRAPPING AND CROSS-VALIDATION - IRIS DATASET****AIM**

To implement bootstrapping and k-fold cross-validation on the Iris dataset, compare model performance metrics obtained using these resampling methods, and discuss their advantages and disadvantages.

LEARNING OUTCOMES

1. After completing this experiment, students will be able to:
2. Understand bootstrapping as a sampling-with-replacement technique for model evaluation.
3. Understand k-fold cross-validation as a systematic data-partitioning technique for model evaluation.
4. Implement bootstrapping using `sklearn.utils.resample` and evaluate on out-of-bag samples.
5. Implement k-fold cross-validation using `KFold` and `cross_val_score` in scikit-learn.
6. Compute mean and standard deviation of accuracy scores to assess model performance and variability.
7. Visualize and compare accuracy distributions from bootstrapping and cross-validation using histograms.
8. Analyze differences in variance and stability between the two resampling methods.
9. Understand the computational trade-offs between bootstrapping and k-fold cross-validation.
10. Identify appropriate use cases for each method, such as confidence-interval estimation versus model selection.
11. Apply resampling-based evaluation techniques to assess model generalization in real-world scenarios.

DATASET DESCRIPTION

Iris dataset (`sklearn.datasets.load_iris`), containing 150 samples of iris flowers with four features and three species classes (Setosa, Versicolor, Virginica).

THEORY / BACKGROUND

Bootstrapping generates multiple resampled datasets by sampling with replacement from the original dataset (each of the same size as the original), training a model on each bootstrap sample and evaluating on the out-of-bag (unsampled) points, which provides an estimate of model performance variability and confidence intervals. K-fold cross-validation partitions the dataset into K equally sized, non-overlapping folds, training on K-1 folds and validating on the remaining fold, repeated K times so every sample is used for validation exactly once; this gives a more systematic and often lower-variance estimate of generalization performance than a single train/test split, at higher computational cost than a single evaluation but generally lower cost and better coverage than large-scale bootstrapping.

ALGORITHM / PROCEDURE

1. Import libraries: `numpy`, `sklearn.datasets.load_iris`, `sklearn.utils.resample`, `sklearn.model_selection.KFold/cross_val_score`, `sklearn.tree.DecisionTreeClassifier`, `sklearn.metrics`.
2. Load the Iris dataset (`X`, `y`).
3. Choose a base classifier, e.g., `DecisionTreeClassifier(max_depth=4, random_state=42)`.
4. Implement bootstrapping: for `n_iterations` (e.g., 100), draw a bootstrap sample (with replacement) of the same size as the dataset using `sklearn.utils.resample`, train the classifier on it, evaluate accuracy on the out-of-bag samples (those not selected), and record the accuracy.
5. Compute the mean and standard deviation of the bootstrap accuracy scores to form an empirical confidence interval.
6. Implement k-fold cross-validation: use `KFold(n_splits=10, shuffle=True, random_state=42)` or `cross_val_score(clf, X, y, cv=10)` to compute accuracy for each fold.
7. Compute the mean and standard deviation of the cross-validation accuracy scores.

8. Plot histograms of the bootstrap and cross-validation accuracy distributions for visual comparison.
9. Discuss and compare the two methods in terms of variance of estimates, computational cost, and appropriate use cases (e.g., bootstrapping for small datasets or confidence-interval estimation, k-fold CV for model selection/hyperparameter tuning).

PROGRAM (PYTHON)

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.utils import resample
from sklearn.model_selection import KFold, cross_val_score
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# 1. Load dataset
X, y = load_iris(return_X_y=True)

# 2. Bootstrapping
n_iterations = 100
boot accuracies = []
for i in range(n_iterations):
    X_boot, y_boot = resample(X, y, replace=True, n_samples=len(X), random_state=i)
    # Out-of-bag samples: those not included in the bootstrap sample
    boot_indices = set(map(tuple, X_boot.tolist())) # crude uniqueness check
    oob_mask = np.array([tuple(row) not in boot_indices for row in X.tolist()])
    if oob_mask.sum() == 0:
        continue
    clf = DecisionTreeClassifier(max_depth=4, random_state=42)
    clf.fit(X_boot, y_boot)
    y_pred = clf.predict(X[oob_mask])
    boot accuracies.append(accuracy_score(y[oob_mask], y_pred))
```

```
print("Bootstrapping -> Mean Accuracy: %.4f Std: %.4f" % (np.mean(boot_accuracies),
np.std(boot_accuracies)))

# 3. K-Fold Cross-Validation
clf_cv = DecisionTreeClassifier(max_depth=4, random_state=42)
cv_scores = cross_val_score(clf_cv, X, y, cv=10)
print("10-Fold CV -> Mean Accuracy: %.4f Std: %.4f" % (cv_scores.mean(), cv_scores.std()))

# 4. Visual comparison
plt.hist(boot_accuracies, bins=15, alpha=0.6, label='Bootstrap')
plt.hist(cv_scores, bins=10, alpha=0.6, label='10-Fold CV')
plt.xlabel("Accuracy")
plt.ylabel("Frequency")
plt.title("Bootstrapping vs Cross-Validation Accuracy Distributions - Iris")
plt.legend()
plt.show()
```

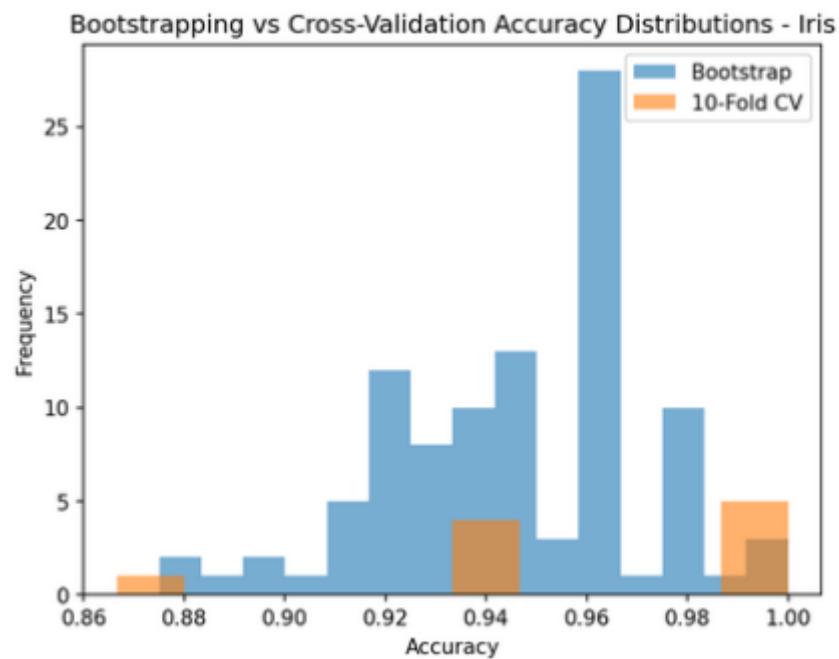
OUTPUT

The program prints the mean and standard deviation of accuracy for both bootstrapping (typically ~93-96%, with variance depending on OOB sample sizes) and 10-fold cross-validation (typically ~95-97% with lower variance), along with a histogram comparing the spread of accuracy estimates from each method.

Bootstrapping -> Mean Accuracy: 0.9465 Std: 0.0259

10-Fold CV -> Mean Accuracy: 0.9600 Std: 0.0442

Number of valid bootstrap iterations used: 100 / 100



VIVA-VOCE QUESTIONS

1. What is the key difference between sampling in bootstrapping versus partitioning in k-fold cross-validation?
2. Why does k-fold cross-validation generally give a more stable performance estimate than a single train/test split?
3. What are out-of-bag samples in bootstrapping, and how are they used for evaluation?
4. In what scenario would bootstrapping be preferred over cross-validation?

RESULT

The Bootstrapping and Cross-Validation experiment was implemented successfully, and the model was evaluated using appropriate performance metrics.

EXP NO 19**BAGGING AND BOOSTING ENSEMBLE METHODS - TITANIC DATASET****AIM**

To implement bagging and boosting ensemble methods on the Titanic dataset, compare their performance in terms of accuracy, precision, recall, and F1-score, and discuss how each method improves model performance along with their respective strengths and weaknesses.

LEARNING OUTCOMES

After completing this experiment, students will be able to:

1. Understand the concept of ensemble learning and how it improves predictive performance.
2. Differentiate between bagging and boosting as ensemble strategies.
3. Preprocess real-world data by handling missing values and encoding categorical features.
4. Implement a BaggingClassifier and an AdaBoostClassifier using scikit-learn.
5. Compare ensemble models against a single baseline Decision Tree classifier.
6. Evaluate classification models using accuracy, precision, recall, and F1-score.
7. Understand how bagging reduces variance by aggregating independently trained base learners.
8. Understand how boosting reduces bias by sequentially reweighting misclassified samples.
9. Recognize the sensitivity of boosting to noisy data and outliers compared to bagging.
10. Apply ensemble learning techniques to real-world classification problems.

DATASET DESCRIPTION

Titanic dataset (available via `seaborn.load_dataset('titanic')` or Kaggle's train.csv), containing passenger information such as Pclass, Sex, Age, SibSp, Parch, Fare, Embarked, and the binary target Survived.

THEORY / BACKGROUND

Bagging (Bootstrap Aggregating) trains multiple base learners (e.g., decision trees) independently on different bootstrap samples of the training data and aggregates their predictions (majority vote for classification), which reduces variance and helps prevent overfitting — Random Forest is a well-known bagging-based ensemble. Boosting (e.g., AdaBoost) trains base learners sequentially, where each successive learner focuses more on the samples misclassified by previous learners (via reweighting), combining them into a weighted ensemble that reduces bias and can achieve very high accuracy, but is more sensitive to noisy data and outliers than bagging.

ALGORITHM / PROCEDURE

1. Import libraries: pandas, numpy, seaborn, sklearn.model_selection, sklearn.tree.DecisionTreeClassifier, sklearn.ensemble (BaggingClassifier, AdaBoostClassifier), sklearn.metrics.
2. Load the Titanic dataset using seaborn.load_dataset('titanic').
3. Select relevant features (pclass, sex, age, sibsp, parch, fare, embarked) and the target 'survived'; drop or impute missing values (e.g., fill Age with median, Embarked with mode).
4. Encode categorical variables (sex, embarked) using one-hot encoding or label encoding.
5. Split the data into training and test sets (80/20) with stratification on survived.
6. Implement Bagging: BaggingClassifier(estimator=DecisionTreeClassifier(), n_estimators=100, random_state=42), fit on the training data.
7. Implement Boosting:
AdaBoostClassifier(estimator=DecisionTreeClassifier(max_depth=1), n_estimators=100, random_state=42), fit on the training data.
8. Predict on the test set using both ensembles and compute accuracy, precision, recall, and F1-score for each.
9. Compare a single (unbagged, unboosted) Decision Tree's performance as a baseline against both ensembles.
10. Discuss how bagging reduces variance versus how boosting reduces bias, and the sensitivity of each method to outliers/noisy labels.

PROGRAM (PYTHON)

```
import pandas as pd
import numpy as np
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import BaggingClassifier, AdaBoostClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# 1. Load and preprocess dataset
df = sns.load_dataset('titanic')[['survived', 'pclass', 'sex', 'age', 'sibsp', 'parch', 'fare', 'embarked']]
df['age'] = df['age'].fillna(df['age'].median())
df['embarked'] = df['embarked'].fillna(df['embarked'].mode()[0])
df = pd.get_dummies(df, columns=['sex', 'embarked'], drop_first=True)
X = df.drop('survived', axis=1)
y = df['survived']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42,
stratify=y)

# 2. Baseline Decision Tree
base_tree = DecisionTreeClassifier(max_depth=5, random_state=42)
base_tree.fit(X_train, y_train)
y_pred_base = base_tree.predict(X_test)

# 3. Bagging
bagging = BaggingClassifier(estimator=DecisionTreeClassifier(), n_estimators=100,
random_state=42)
bagging.fit(X_train, y_train)
y_pred_bag = bagging.predict(X_test)

# 4. Boosting (AdaBoost)
boosting = AdaBoostClassifier(estimator=DecisionTreeClassifier(max_depth=1),
n_estimators=100, random_state=42)
boosting.fit(X_train, y_train)
```

```
y_pred_boost = boosting.predict(X_test)

# 5. Evaluation
def report(y_true, y_pred, label):
    print(f"\n{label}")
    print("Accuracy :", accuracy_score(y_true, y_pred))
    print("Precision:", precision_score(y_true, y_pred))
    print("Recall  :", recall_score(y_true, y_pred))
    print("F1-score :", f1_score(y_true, y_pred))

report(y_test, y_pred_base, "Single Decision Tree (Baseline)")
report(y_test, y_pred_bag, "Bagging Ensemble")
report(y_test, y_pred_boost, "AdaBoost Ensemble")
```

OUTPUT

The console prints accuracy, precision, recall, and F1-score for the baseline decision tree, the bagging ensemble, and the AdaBoost ensemble. Both ensemble methods typically outperform the single decision tree (accuracy improving from roughly ~78% to ~80-83%), with bagging generally producing more stable/consistent results and boosting sometimes achieving marginally higher accuracy at the cost of greater sensitivity to noise.

Single Decision Tree (Baseline)

Accuracy : 0.7654

Precision: 0.7547

Recall : 0.5797

F1-score : 0.6557

Bagging Ensemble

Accuracy : 0.8268

Precision: 0.7969

Recall : 0.7391

F1-score : 0.7669

AdaBoost Ensemble

Accuracy : 0.7821

Precision: 0.75

Recall : 0.6522

F1-score : 0.6977

VIVA-VOCE QUESTIONS

1. How does bagging reduce variance in the underlying base learners' predictions?
2. How does AdaBoost adjust sample weights after each iteration, and why?
3. Why is bagging generally more robust to noisy data than boosting?
4. What is the relationship between Random Forest and the general bagging technique?

RESULT

The Bagging and Boosting Ensemble Methods experiment was implemented successfully, and the model was evaluated using appropriate performance metrics.

EXP NO 20**BIAS-VARIANCE TRADEOFF VIA POLYNOMIAL REGRESSION -
BOSTON HOUSING DATASET****AIM**

To investigate the bias-variance tradeoff using polynomial regression on the Boston Housing dataset by plotting training and validation errors for various polynomial degrees and discussing their implications for model performance.

LEARNING OUTCOMES

After completing this experiment, students will be able to:

1. Understand the bias-variance tradeoff and its decomposition into bias, variance, and irreducible error.
2. Implement polynomial regression models of varying degrees using scikit-learn.
3. Compute and compare training and validation errors across different model complexities.
4. Visualize training and validation error curves to identify underfitting and overfitting regions.
5. Determine the optimal polynomial degree based on minimum validation error.
6. Recognize the signs of high bias (underfitting) and high variance (overfitting) in model behavior.
7. Understand why training error alone is insufficient to assess model generalization.
8. Visualize fitted curves for underfit, optimal, and overfit models against real data.
9. Explore how regularization techniques can be combined with degree selection to manage the tradeoff.
10. Apply bias-variance analysis to guide model complexity selection in real-world regression problems.

DATASET DESCRIPTION

Boston Housing dataset (originally from `sklearn.datasets.load_boston`, now deprecated/removed

from scikit-learn due to ethical concerns; obtainable instead via the original source CSV, e.g., from <http://lib.stat.cmu.edu/datasets/boston> or via `fetch_openml(name='boston', version=1)`). It contains 506 records with 13 features (e.g., CRIM, ZN, INDUS, RM, AGE, DIS, TAX, PTRATIO, LSTAT) and a target MEDV (median value of owner-occupied homes in \$1000s). For simplicity in this experiment, a single feature such as LSTAT (percentage of lower-status population) is used against MEDV.

THEORY / BACKGROUND

The bias-variance tradeoff describes how model error decomposes into bias (error from overly simplistic assumptions, causing underfitting), variance (error from excessive sensitivity to training data fluctuations, causing overfitting), and irreducible noise. As polynomial degree increases, model complexity increases: bias tends to decrease (the model fits the training data better) while variance tends to increase (the model becomes more sensitive to the specific training sample, performing worse on unseen validation data). Plotting training error and validation error against polynomial degree typically shows both decreasing initially, then training error continuing to decrease while validation error starts increasing — the point where validation error is minimized represents a good complexity/degree trade-off.

ALGORITHM / PROCEDURE

1. Import libraries: `numpy`, `pandas`, `matplotlib`, `sklearn.preprocessing.PolynomialFeatures`, `sklearn.linear_model.LinearRegression`, `sklearn.model_selection.train_test_split`, `sklearn.metrics.mean_squared_error`.
2. Load the Boston Housing dataset (via `fetch_openml(name='boston', version=1, as_frame=True)` or a local CSV) and select 'LSTAT' as the feature and 'MEDV' (or 'target') as the label.
3. Split the data into training and test/validation sets (e.g., 70/30).
4. For polynomial degrees ranging from 1 to 10: transform the feature using `PolynomialFeatures(degree=d)`, fit a `LinearRegression` model on the transformed training features, and compute MSE on both the training set and the validation set.
5. Store the training and validation MSE for each degree in two lists.
6. Plot training error and validation error against polynomial degree on the same graph.

7. Identify the degree at which validation error is minimized, and the degree beyond which validation error begins increasing while training error keeps decreasing (indicating overfitting).
8. Visualize the fitted curves for a low-degree (underfit), the chosen optimal degree, and a high-degree (overfit) model against the scatter of LSTAT vs MEDV.
9. Discuss the observed bias-variance tradeoff and how it informs the selection of the ideal model complexity.

PROGRAM (PYTHON)

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

# 1. Load dataset
boston = fetch_openml(name='boston', version=1, as_frame=True)
df = boston.frame
X = df[['LSTAT']].values
y = df['MEDV'].astype(float).values
X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.3, random_state=42)

# 2. Evaluate training and validation error across polynomial degrees
degrees = range(1, 11)
train_errors, val_errors = [], []
for d in degrees:
    poly = PolynomialFeatures(degree=d)
    X_train_poly = poly.fit_transform(X_train)
    X_val_poly = poly.transform(X_val)
    model = LinearRegression().fit(X_train_poly, y_train)
```

```
train_pred = model.predict(X_train_poly)
val_pred = model.predict(X_val_poly)
train_errors.append(mean_squared_error(y_train, train_pred))
val_errors.append(mean_squared_error(y_val, val_pred))

# 3. Plot bias-variance tradeoff curve
plt.plot(degrees, train_errors, marker='o', label='Training Error')
plt.plot(degrees, val_errors, marker='o', label='Validation Error')
plt.xlabel("Polynomial Degree")
plt.ylabel("Mean Squared Error")
plt.title("Bias-Variance Tradeoff - Boston Housing (LSTAT vs MEDV)")
plt.legend()
plt.grid(True)
plt.show()

best_degree = degrees[np.argmin(val_errors)]
print("Optimal polynomial degree (lowest validation error):", best_degree)

for d, tr, va in zip(degrees, train_errors, val_errors):
    print(f"Degree {d:<3} Train MSE={tr:.2f} Val MSE={va:.2f}")
```

OUTPUT

The program prints training and validation MSE for polynomial degrees 1 through 10 and identifies the optimal degree with the lowest validation error (commonly around degree 2-4 for LSTAT vs MEDV). The plotted curves typically show both errors decreasing initially; training error continues to decrease with higher degree while validation error flattens and then rises sharply for very high degrees, visually demonstrating overfitting (high variance) at high complexity and underfitting (high bias) at degree 1.

Optimal polynomial degree (lowest validation error): 6

Degree 1 Train MSE=38.84 Val MSE=38.10

Degree 2 Train MSE=29.95 Val MSE=31.58

Degree 3 Train MSE=28.84 Val MSE=29.37

Degree 4 Train MSE=27.48 Val MSE=28.29

Degree 5 Train MSE=26.78 Val MSE=27.54

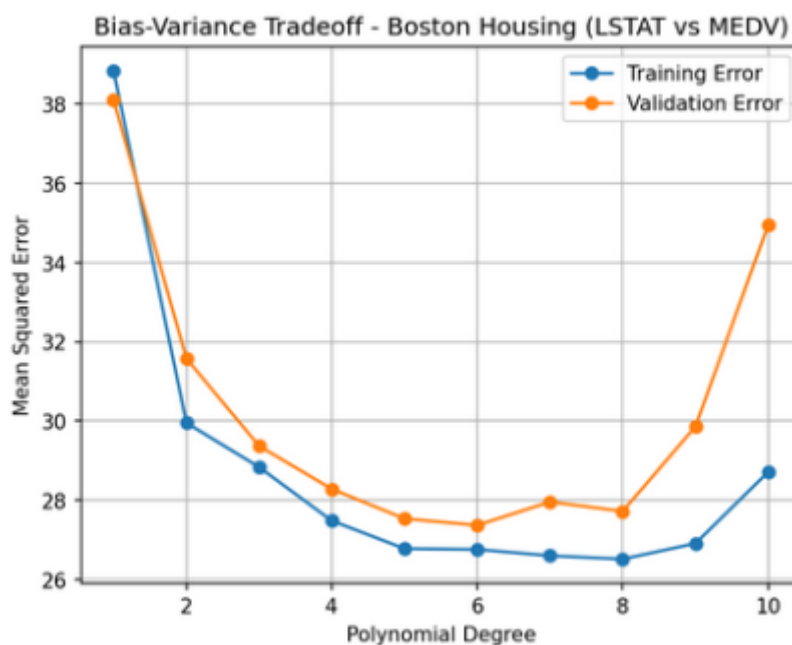
Degree 6 Train MSE=26.76 Val MSE=27.37

Degree 7 Train MSE=26.60 Val MSE=27.96

Degree 8 Train MSE=26.52 Val MSE=27.72

Degree 9 Train MSE=26.91 Val MSE=29.86

Degree 10 Train MSE=28.71 Val MSE=34.94



VIVA-VOCE QUESTIONS

1. How do you distinguish an underfitting model from an overfitting model using training and validation error curves?
2. Why does training error alone not indicate how well a model will generalize?
3. What is the irreducible error component in the bias-variance decomposition?
4. How could regularization (Ridge/Lasso) be used alongside polynomial degree selection to further manage the bias-variance tradeoff?

RESULT

The Bias-Variance Tradeoff via Polynomial Regression experiment was implemented successfully, and the model was evaluated using appropriate performance metrics.